

INFORME FINAL — ERSO 2026-008

- [INFORME FINAL — ERSO 2026-008](#)
 - [Ciclo de vida y trazabilidad de versiones del DID Document](#)
 - [Cómo leer este informe](#)
 - [Contenido](#)
 - [Resumen ejecutivo](#)
- [PARTE I — MARCO CONCEPTUAL: los términos que necesitas](#)
 - [1. La historia en cinco minutos](#)
 - [2. Diccionario de términos](#)
 - [3. Las siete ideas madre](#)
 - [4. El método de trabajo: cómo se piensa un criterio](#)
- [PARTE II — QUÉ PIDE LA ERSO 008, EXPLICADO FRASE POR FRASE](#)
 - [5. La ficha del documento](#)
 - [6. Capacidades del proceso](#)
 - [7. Condiciones del proceso](#)
 - [8. Descripción del proceso](#)
 - [9. Qué debe hacer: los ocho pasos](#)
 - [10. Los siete criterios de aceptación](#)
- [PARTE III — EL PROYECTO: QUÉ SE CONSTRUYÓ PARA ESTA ERSO Y CÓMO ESTÁ MONTADO](#)
 - [11. Visión general](#)
 - [12. El entorno para esta ERSO: el protocolo «a mano»](#)
 - [13. Cómo funciona por dentro \(lo mínimo para entender las pruebas\)](#)
- [PARTE IV — LABORATORIOS DE DEMOSTRACIÓN: LA PRUEBA MATERIAL DE CADA CRITERIO](#)
 - [14. Cómo funcionan los laboratorios](#)
 - [15. Preparación común](#)
 - [16. Mapa de los laboratorios](#)
 - [17. CRITERIO 1 — Las precondiciones se cumplen; si alguna falla, no se emite desafío](#)
 - [18. CRITERIO 2 — El desafío es de un solo uso, con tipo y audiencia específicos](#)
 - [19. CRITERIO 3 — El documento no incluye claves privadas, y `id` y controladores corresponden al recurso autorizado](#)
 - [20. CRITERIO 4 — La escritura exige la versión esperada y devuelve versión, hash y URL pública](#)
 - [21. CRITERIO 5 — La publicación se confirma por lectura de la URL y comparación de hash](#)
 - [22. CRITERIO 6 — Si no hay respuesta en plazo, la operación queda pendiente y no se da por publicada](#)
 - [23. CRITERIO 7 — La traza permite reconstruir el estado en cada momento](#)
 - [24. Pruebas automáticas](#)
- [PARTE V — PREGUNTAS Y RESPUESTAS](#)
 - [25. Preguntas sobre los conceptos](#)
 - [26. Preguntas por laboratorio y por criterio](#)
- [PARTE VI — CÓMO RESPONDER A CADA CRITERIO](#)
 - [27. Redacción modelo de las siete respuestas](#)
 - [28. Lista de comprobación para quien acepta](#)
- [PARTE VII — LÍMITES, HALLAZGOS Y OPERACIÓN](#)
 - [29. Lo que este informe NO demuestra, y lo que se encontró](#)
 - [30. Si algo no sale como en el informe](#)
 - [31. Operación diaria](#)
- [ANEXOS](#)
 - [Anexo A — Los scripts](#)
 - [Anexo B — Autocomprobación \(con respuestas\)](#)

- [Anexo C — Ejercicios del método \(con respuesta modelo\)](#)
- [Anexo D — Referencias](#)

INFORME FINAL — ERSO 2026-008

Ciclo de vida y trazabilidad de versiones del DID Document

Manual de comprensión, construcción y comprobación

Proyecto	Identidad Digital Soberana SSI
ERSo	2026-008 — Ciclo de vida y trazabilidad de versiones del DID Document
Desarrollador asignado (según la ERSo)	Geovani Rincón
Plantilla de pruebas y pruebas funcionales (según la ERSo)	Luis González
Fechas de la ERSo	Elaboración: viernes 18-sep-2026 · Desarrollo: miércoles 23-sep-2026, 7:30 a. m. · Prueba: viernes 25-sep-2026, 7:30 a. m.
Dominio del laboratorio	civica-desarrollo.avance.org.co
Entidad de ejemplo	avance-issuer (Avance, emisor institucional) · espacio entidades/...
Ubicación del proyecto	/home/geovani/Descargas/generic/bitacora/new_proyect
Salidas de terminal de este informe	Obtenidas ejecutando los comandos reales sobre el proyecto, el 1 de octubre de 2026
Complementos	scripts/lab-008-completo.sh (todos los laboratorios en un script) · informes/ERSo-2026-008.md (versión corta) · informes finales de las ERSo 004 a 007

Nota de fechas. La fecha de prueba que fija la ERSo (25-sep-2026) es anterior a la fecha de este informe (1-oct-2026). El documento sirve tanto para repetir la prueba como para dejar constancia de ella; las firmas de la tabla de actividades son de las personas responsables.

Cómo leer este informe

Está escrito para que **cualquier persona** pueda entender qué se pidió, qué se construyó y cómo se comprobó, **sin ser experta**. Cada término técnico se explica la primera vez que aparece; la Parte I es un diccionario.

Símbolo	Significa
💡	Idea clave: lo más importante de la sección, en una frase
🔍	Qué vas a ver: lo que debes esperar <i>antes</i> de ejecutar un comando
✅	Qué significa: cómo interpretar lo que el sistema respondió

Símbolo	Significa
⚠	Cuidado: una trampa frecuente o un límite que conviene conocer
✍	Predice antes de ejecutar: una pregunta para que pienses antes de ver el resultado

Si eres...	Lee esto
Quien decide o aprueba	El Resumen ejecutivo y la Parte VII
Quien necesita entender el problema	Parte I y Parte II
Quien debe reproducir y comprobar	Partes III y IV
Quien prepara la prueba funcional	Partes IV, VI y la lista de comprobación
Quien prepara una exposición	Partes V y VI

Cómo están presentados los comandos. Los bloques `console` son una **sesión de terminal real**: las líneas con \$ son lo que se escribe; las demás, lo que el sistema respondió. Los valores que cambian en cada ejecución (horas, identificadores, claves, huellas) serán distintos al repetir; lo que debe coincidir es el **patrón** explicado debajo de cada salida.

Esta ERSO es la más completa de la serie. Reúne todo lo anterior —canales (004), documento (005), entidad (006), verificación (007)— y añade **el tiempo**: un documento que se crea, cambia y se da de baja, y cuya historia se puede reconstruir. Por eso, a diferencia de las anteriores, aquí **el protocolo de escritura se ejecuta a mano, paso por paso**, para ver cada puerta que cruza una operación.

Contenido

Parte	Secciones	De qué trata
Resumen ejecutivo	—	El problema, la solución y el resultado en una página
I · Marco conceptual	1 a 4	Los términos, en lenguaje sencillo, y el método de trabajo
II · Qué pide la ERSO	5 a 10	Capacidades, condiciones, descripción, pasos y criterios, frase por frase
III · El proyecto	11 a 13	Qué se construyó, cómo se monta en Docker y cómo funciona por dentro
IV · Laboratorios	14 a 24	La prueba material de cada criterio, con cada comando y su respuesta
V · Preguntas y respuestas	25 y 26	Todas las preguntas de comprensión con su respuesta
VI · Cómo responder	27 y 28	Redacción modelo de cada criterio y lista de comprobación
VII · Límites y operación	29 a 31	Lo que NO se demuestra, qué hacer si algo falla y operación diaria
Anexos	A a D	Scripts, autocomprobación, ejercicios y referencias

Resumen ejecutivo

El problema. Un DID Document institucional no es una foto fija: la institución **rota sus claves**, **añade servicios** y, un día, **deja de existir** o es dada de baja. Si cualquiera pudiera cambiar el documento, o si un reintento por un corte de red creara versiones duplicadas, o si una escritura simultánea pisara a otra, la identidad dejaría de ser confiable. Además, ante una auditoría, hay que poder responder: «¿qué decía el documento el 1 de octubre a las 17:26?».

La solución. Cada operación (crear, actualizar, desactivar) atraviesa **una cadena de puertas** en un orden definido:

1. **Precondiciones** (cuenta, espacio propio, canal): si fallan, ni siquiera se entrega un desafío.
2. **Desafío** de un solo uso, con tipo de operación y audiencia.
3. **Documento** sin secretos y con `id` y controladores correctos.
4. **Prueba de posesión**: firma sobre *desafío* + *huella del documento* con la clave **vigente**.
5. **Escritura condicionada** a la versión esperada (If-Match) y con clave de idempotencia.
6. **Confirmación** leyendo la URL pública y comparando huella.
7. Si no hay respuesta a tiempo: **pendiente**, y luego **reconciliar**.
8. Todas las versiones se **conservan** en una tabla que no admite modificaciones.

El resultado.

#	Criterio	Resultado	Dónde
1	Precondiciones; si alguna falla, no se emite desafío	✓	§17
2	Desafío de un solo uso, con tipo y audiencia	✓	§18
3	Documento sin claves privadas; <code>id</code> y controladores correctos	✓	§19
4	Escritura con versión esperada; devuelve versión, hash y URL	✓	§20
5	Publicación confirmada por lectura de URL y comparación de hash	✓	§21
6	Sin respuesta en plazo: pendiente y no se da por publicada	✓	§22
7	La traza permite reconstruir el estado en cada momento	✓	§23

Cifras: 8 pruebas automáticas propias, todas en verde; 7 criterios demostrados con el protocolo ejecutado a mano; un historial real de 4 versiones (crear, rotar clave, añadir servicio, desactivar) reconstruido instante por instante.

Lo que debes saber de antemano: (1) la ERSO advierte que *rotar el documento no da por sí solo validación histórica* y que *la recuperación ante pérdida de clave no se resuelve guardando un documento público*: ambas se tratan como **límites explícitos** (§29); (2) para demostrar el criterio 6 hubo que levantar una **segunda instancia temporal** del registro con una dirección de confirmación inalcanzable (§22).

PARTE I — MARCO CONCEPTUAL: los términos que necesitas

1. La historia en cinco minutos

Piensa en una **escritura pública** en una notaría.

- Para **registrar** una escritura nueva, hay que identificarse (precondiciones), recibir un número de turno de un solo uso (desafío) y firmar delante del notario (prueba de posesión).
- Para **modificarla**, hay que decir sobre **qué versión** se trabaja: si alguien la modificó mientras tanto, el notario se niega (If-Match).
- Si la secretaría repite el trámite por un malentendido, el notario reconoce «esto ya lo hicimos» y no crea otro (idempotencia).
- Cuando se **da de baja**, queda en el libro marcada como cancelada, **para siempre**: no se resucita; si hace falta, se abre una escritura nueva.
- Antes de dar por registrado el trámite, el notario **va a la ventanilla pública y comprueba** que el documento expuesto es idéntico (confirmación). Si la ventanilla no responde, el trámite queda **pendiente**; no «hecho».
- Y el **libro de registro** es de tinta indeleble: se puede preguntar «¿qué decía esa escritura el 1 de octubre a las 17:26?» y responder con exactitud.

💡 Esta ERSO es **la vida completa de un documento**: nacer, cambiar, morir y dejar rastro.

2. Diccionario de términos

2.1 Los de siempre (resumen)

Término	En palabras sencillas
DID / <code>did:web</code>	Identificador que se convierte en una dirección web: <code>did:web:ejemplo.org:entidades:avance</code> → <code>https://ejemplo.org/entidades/avance/did.json</code> .
DID Document	El JSON público del DID: claves públicas y para qué sirven.
Clave pública / privada	La privada firma y no se comparte; la pública verifica.
Hash (sha256:...)	Huella digital de un contenido; cambia por completo si se altera un carácter.
JSON canónico	JSON escrito siempre igual (claves ordenadas, sin espacios) para que la huella sea estable.
mTLS	Conexión donde ambos lados muestran certificado.
Namespace	La «repisa» reservada a una entidad (<code>entidades/...</code>).
JWT / token	Pase firmado de corta duración (10 min) que dice qué puede hacer una entidad.

Término	En palabras sencillas
Desafío (challenge)	Número aleatorio de un solo uso, que se entrega para que el cliente pruebe que posee la clave.
Prueba de posesión	Firma (JWS) del cliente sobre el desafío, hecha con la clave del DID.
VDR	El registro que guarda y entrega los DID Document.
Auditoría	Libro que no se puede editar ni borrar.

2.2 El ciclo de vida

Término	En palabras sencillas
Ciclo de vida	Las etapas por las que pasa un documento: crear, actualizar (las veces que haga falta) y desactivar.
CREATE	Primera publicación. Espera «versión 0» (no existe).
UPDATE	Cambio de un documento que ya existe. Genera una versión nueva.
DEACTIVATE	Baja. No lleva documento nuevo.
Versión	Número consecutivo (1, 2, 3...) de cada estado del documento. Las anteriores no se borran .
Estado ACTIVE / DEACTIVATED	Si el DID está vigente o dado de baja.
Estado terminal	Un estado del que no se vuelve. DEACTIVATED lo es: no se reactiva; se necesita una instancia nueva (otro namespace).
Rotación de claves	Cambiar la clave del documento por otra nueva. Es un UPDATE especial: la prueba la firma la clave vieja (la vigente) y el documento nuevo lleva la nueva .
Clave vigente	La que figura en la versión actual del documento.
Controlador	Quién está autorizado a gobernar el documento (aquí, el propio DID).

2.3 Escribir con seguridad

Término	En palabras sencillas
Precondición	Algo que debe cumplirse <i>antes</i> de empezar. Si falla, no se avanza.
Tipo del desafío	Para qué operación se pidió: CREATE , UPDATE o DEACTIVATE . Un desafío de un tipo no sirve para otro.
Audiencia (aud)	Para quién es el desafío: <code>vdr:<dominio>:did-operation</code> . Impide usar un desafío de un sistema en otro.
Un solo uso	Un desafío se gasta al primer intento, salga bien o mal.
Vigencia (TTL)	Cuánto dura un desafío (300 s). Pasado ese tiempo, caduca.
Vincular la firma al documento	La prueba firma el desafío (que es de ahora) y la huella del documento (que es de este contenido). Así no se puede reutilizar para otro contenido.
Control de concurrencia optimista (If-Match)	«Solo escribo si la versión actual es N». Si otro cambió mientras tanto, se rechaza (412). Evita que dos escritores se pisen.

Término	En palabras sencillas
ETag	La versión del documento, expresada como cabecera HTTP ("2").
Idempotencia (Idempotency-Key)	Una clave que identifica una <i>intención</i> . Repetir la misma petición con la misma clave devuelve el resultado original sin duplicar.
Repetición (replayed)	Marca en la respuesta que indica «esta es la respuesta de antes, no una operación nueva».

2.4 Confirmar y reconciliar

Término	En palabras sencillas
Confirmación	Tras escribir, el registro lee su propia URL pública y compara la huella.
CONFIRMED	La lectura coincidió. La publicación se da por hecha.
PENDING (pendiente)	No se pudo confirmar a tiempo. No se da por publicada. (HTTP 202 : «aceptado, pero no terminado»).
Reconciliar	Volver a leer la URL para resolver una operación pendiente: pasa a CONFIRMED si coincide (o SUPERSEDED si, entre tanto, hay una versión posterior).

2.5 Traza

Término	En palabras sencillas
Traza / historial de versiones	La secuencia de todas las versiones, con quién, cuándo y qué huella.
Solo-agregar (append-only)	Tabla donde solo se pueden añadir filas; modificar o borrar está prohibido por la propia base de datos (disparadores).
Reconstruir el estado	Dado un instante, decir cuál era la versión vigente y su estado en ese momento.
Auditoría de eventos	Registro de cada acción (CHALLENGE_ISSUED , WRITE_UPDATE , PUBLICATION_CONFIRMED ...).
Validación histórica	Verificar una firma con la clave de su época . Advertencia: no se ofrece (§29).

2.6 Códigos HTTP que aparecerán

Código	Significa aquí
200 / 201	Éxito (201 al crear)
202	Aceptado pero pendiente de confirmar
400	Petición rechazada por nginx (falta certificado)
403	Desafío/prueba inválidos o no permitidos
404	No existe
409	Estado terminal (TERMINAL_STATE)
410	Gone: el DID está desactivado
412	Precondición fallida (de la entidad o de versión)
422	Documento no válido, o clave de idempotencia reutilizada

Código	Significa aquí
428	Falta una cabecera obligatoria (If-Match)

3. Las siete ideas madre

1. **Todo cambio tiene un camino:** desafío → prueba → escritura → confirmación.
2. **La firma ata el momento y el contenido:** desafío (cuándo) + huella (qué).
3. **Quien firma es quien tiene la llave hoy:** la clave vigente, no la del documento nuevo.
4. **Nadie pisa a nadie:** If-Match y bloqueo de la fila del DID.
5. **Repetir no duplica:** idempotencia.
6. **Hecho es hecho solo si se comprobó:** confirmación; si no, pendiente.
7. **El pasado no se reescribe:** versiones y auditoría de solo-agregar; la baja es terminal.

4. El método de trabajo: cómo se piensa un criterio

1. **Descomponer** el criterio en el verbo y la evidencia.
 2. **Hacerlo comprobable:** ¿qué comando lo demuestra?
 3. **Buscar el caso negativo:** casi todo criterio de seguridad se prueba con lo que **debe fallar**.
 4. **Predecir** el resultado.
 5. **Ejecutar y observar.**
 6. **Correlacionar** con la regla del código que lo causa.
 7. **Redactar** la respuesta con la evidencia.
-

PARTE II — QUÉ PIDE LA ERSo 008, EXPLICADO FRASE POR FRASE

5. La ficha del documento

Campo del PDF	Valor
Proyecto	Identidad Digital Soberana SSI
Título	Ciclo de vida y trazabilidad de versiones del DID Document
Desarrollador	Geovani Rincón
Fecha de la ERSo	Viernes, 18 de septiembre de 2026
Fecha de desarrollo	Miércoles, 23 de septiembre de 2026, 7:30 a. m.
Fecha de prueba	Viernes, 25 de septiembre de 2026, 7:30 a. m.
Responsables	Análisis y diseño: Karen Flórez Madiedo · Asignar desarrollador, aceptar archivos y archivo: Rocío Villamizar · Implementar: Geovani Rincón · Plantilla de pruebas y pruebas funcionales: Luis González

⚠ Las firmas y fechas de la tabla de actividades **corresponden a las personas responsables**; este informe aporta las **evidencias técnicas**, no las firmas.

Capacidades → Condiciones → Descripción → Qué debe hacer (8 pasos) → Criterios (7) → Actividades

6. Capacidades del proceso

6.1 La frase principal, desarmada

"Se requiere habilitar el **ciclo de vida** del DID Document institucional, cubriendo **publicación autenticada, actualización controlada, desactivación y trazabilidad de versiones**, de modo que **la traza permita reconstruir el estado en cada momento**."

#	Fragmento	En lenguaje sencillo	Cómo se materializó
1	" ciclo de vida "	El documento nace, cambia y muere	CREATE · UPDATE · DEACTIVATE
2	" publicación autenticada "	Solo quien prueba quién es publica	mTLS + token + prueba de posesión

#	Fragmento	En lenguaje sencillo	Cómo se materializó
3	" actualización controlada "	Los cambios no se pisan ni se duplican	If-Match + idempotencia
4	" desactivación "	La baja es una operación propia y definitiva	POST /documents/{did}/deactivate
5	" trazabilidad de versiones "	Se conserva cada versión	did_document_versions solo-agregar
6	" reconstruir el estado en cada momento "	Pregunta por un instante, respuesta exacta	GET /documents/{did}/state?at=

6.2 Las seis capacidades

#	Capacidad (texto del PDF)	Qué se busca	Cómo se resolvió	Pasos
1	<i>Publicación autenticada del documento</i>	Crear con identidad probada	Escritura CREATE	2-6
2	<i>Actualización controlada del documento</i>	Cambiar sin pisar a nadie	UPDATE con versión esperada	3-5
3	<i>Desactivación mediante operación específica sobre el recurso</i>	Una baja explícita	DEACTIVATE (ruta propia)	2, 4, 5
4	<i>Trazabilidad de versiones</i>	Historia consultable	Versiones + auditoría	8
5	<i>Comprobación de posesión mediante desafío de un solo uso, con tipo y audiencia específicos</i>	Probar que se tiene la clave ahora	Desafío + JWS ES256	2, 4
6	<i>Confirmación mediante lectura de la URL calculada y comparación de contenido y hash</i>	No creerse a uno mismo	confirm()	6, 7

7. Condiciones del proceso

Son tres bloques.

7.1 Precondiciones

"Cuenta de entidad habilitada con namespace único, ruta did:web reservada, perfil y métodos de verificación definidos, y canal de escritura restringido con lectura pública por canal separado."

Precondición	Qué significa	Código si falla
Cuenta habilitada	La entidad existe y no está deshabilitada	ACCOUNT_MISSING_OR_DISABLED
Namespace único	La ruta tiene un solo dueño	NAMESPACE_NOT_OWNED
Ruta reservada	Alguien apartó esa ruta	NAMESPACE_NOT_RESERVED
Perfil y métodos definidos	El espacio tiene reglas (qué tipos de clave, cuántas)	PROFILE_UNDEFINED

Precondición	Qué significa	Código si falla
Canal restringido	Llega con certificado válido de la misma entidad	WRITE_CHANNEL_NOT_RESTRICTED
Lectura pública separada	Otra puerta (ERSo 004/006)	—

7.2 Condiciones por operación

"Cada paso se valida con una única condición."

Paso	Condición única
Emitir desafío	Precondiciones cumplidas (si no, no se emite)
Desafío	De un solo uso, con tipo y audiencia
Documento	Sin claves privadas; <code>id</code> y controladores correctos
Prueba	Firma sobre desafío y hash con clave <code>authentication</code> vigente
Escritura	Versión esperada + clave de idempotencia
Confirmación	Lectura de la URL y comparación de hash
Sin respuesta en plazo	Queda pendiente (no «publicada»)

7.3 Regla transversal

"Un historial en estado terminal no vuelve a estado activo sin nueva instancia."

- **Qué significa.** Un DID desactivado no se reactiva. Si la entidad quiere volver, se crea un DID **nuevo** (otra ruta).
- **Qué problema evita.** Que alguien (o un error) «resucite» una identidad dada de baja, con consecuencias para quien confió en la baja.
- **Cómo se cumple.** `409 TERMINAL_STATE` (§23).

8. Descripción del proceso

"El documento publicado no es estático: se crea, se actualiza y se da de baja, y debe poder reconstruirse qué estado tenía en cada momento."

Resume la ERSo: **cambio + memoria**.

"...escritura autenticada, actualización condicionada a la versión esperada, desactivación explícita y una traza de versiones que permita auditar."

Cuatro mecanismos, cada uno con su criterio.

Advertencia importante para el desarrollador: "rotar el documento actual no ofrece por sí solo validación histórica, y la recuperación ante pérdida de clave no se resuelve guardando

un documento público."

Son dos advertencias, y conviene entenderlas bien:

Advertencia	Qué quiere decir	Qué se hizo
Rotar no da validación histórica	Si cambias la clave del documento, el documento nuevo contiene la clave nueva. Una firma antigua, hecha con la clave vieja, ya no se puede verificar mirando solo el documento actual.	Se conserva el historial (para reconstruir el estado de cada instante) pero el consumidor de la ERSO 007 valida contra el documento actual . Queda como límite explícito (§29).
La recuperación ante pérdida de clave no se resuelve guardando un documento público	Si pierdes la clave privada, nada que esté publicado te devuelve el control (el documento es público: cualquiera lo tiene). La recuperación requiere otro mecanismo (custodia, respaldo fuera de línea, autoridad de recuperación).	No se inventó recuperación. La pérdida de clave obliga a desactivar y crear una instancia nueva. Queda como límite explícito.

"Literatura y temas a consultar"

Lectura	Qué te aporta
W3C DID v1.1	Operaciones del método: crear, actualizar, desactivar
DID Resolution v1	Cómo se ve un DID desactivado desde el consumidor
Control de concurrencia optimista (versión esperada / ETag / If-Match)	Pasos 5 y criterio 4
Claves de idempotencia para escrituras	Reintentos seguros
Pruebas de posesión con nonce de un solo uso (challenge-response)	Criterio 2
Revocación / desactivación de identificadores	Estado terminal

9. Qué debe hacer: los ocho pasos

1. Verificar las precondiciones (cuenta de entidad, namespace único, ruta did:web reservada, perfil y métodos definidos, canal de escritura restringido).
2. Solicitar el desafío indicando el propósito (CREATE, UPDATE o DEACTIVATE).
3. Construir o actualizar el documento con el identificador y los controladores del recurso autorizado, sin claves privadas.
4. Firmar la prueba de posesión sobre el desafío y el hash del documento de la operación.
5. Escribir en el registro con la versión esperada y la clave de idempotencia, y recibir versión, hash y URL pública.
6. Confirmar la publicación leyendo el documento desde la URL calculada y comparando contenido y hash.

7. Si no hay respuesta en plazo, dejar la operación en estado pendiente y reconciliar la versión efectivamente publicada.
8. Conservar la secuencia de versiones para reconstruir la traza de cada estado.

Paso	Lo que pide	Criterio que lo comprueba	Laboratorio
1	Precondiciones	Criterio 1	\$17
2	Solicitar desafío con propósito	Criterio 2	\$18
3	Documento sin secretos, con <code>id</code> y controladores	Criterio 3	\$19
4	Firmar sobre desafío y hash	Criterio 2 (y rotación, \$20)	\$18 y \$20
5	Escribir con versión e idempotencia	Criterio 4	\$20
6	Confirmar por lectura y hash	Criterio 5	\$21
7	Pendiente y reconciliar	Criterio 6	\$22
8	Conservar versiones	Criterio 7	\$23

💡 Los ocho pasos y los siete criterios se corresponden casi uno a uno: la ERSO es una **cadena de puertas** y cada criterio comprueba una.

10. Los siete criterios de aceptación

1. Las precondiciones se cumplen; si alguna falla no se emite desafío; evidencia: verificación de cuenta y canal. **2.** El desafío es de un solo uso, con tipo y audiencia específicos de la operación DID; evidencia: registro de emisión del desafío. **3.** El documento no incluye claves privadas y el `id` y los controladores corresponden al recurso autorizado; evidencia: documento de la operación. **4.** La escritura exige la versión esperada y devuelve versión, hash y URL pública; evidencia: respuesta del registro. **5.** La publicación se confirma por lectura de la URL y comparación de hash; evidencia: comparación de contenido y hash. **6.** Si no hay respuesta en plazo, la operación queda pendiente y no se da por publicada; evidencia: estado de la operación. **7.** La traza permite reconstruir el estado en cada momento; evidencia: historial de versiones.

Criterio	Verbo y evidencia	Lo que debo poder mostrar	El caso negativo	Mecanismo
1	Si falla, no se emite · cuenta y canal	Cada precondición incumplida bloquea el desafío	7 casos distintos	<code>preconditionFailures</code>
2	Un solo uso, tipo y audiencia · registro de emisión	El desafío trae tipo y audiencia, y queda en la auditoría	Reutilizar; cambiar de tipo; caducado; audiencia ajena	<code>consumeChallenge</code> + <code>payload</code>
3	No incluye, corresponden · documento de la operación	Documentos prohibidos rechazados	Clave privada, <code>id</code> ajeno, controlador ajeno, datos civiles	Validador <code>PUBLISHER</code>

Criterio	Verbo y evidencia	Lo que debo poder mostrar	El caso negativo	Mecanismo
4	<i>Exige versión</i> · respuesta	428 sin versión; 412 con versión errónea; 200/201 con versión, hash, URL	Sin If-Match ; versión errónea; repetición	If-Match + idempotencia
5	<i>Se confirma</i> · contenido y hash	Hash de la evidencia = hash del documento público	—	confirm()
6	<i>Queda pendiente</i> · estado	Estado PENDING cuando no se puede leer; CONFIRMED tras reconciliar	Lectura imposible	Plazo + reconcile
7	<i>Permite reconstruir</i> · historial	Estado en cada instante, 404 antes de existir, tablas inalterables	Intentar modificar el pasado	state?at= + disparadores

PARTE III — EL PROYECTO: QUÉ SE CONSTRUYÓ PARA ESTA ERSO Y CÓMO ESTÁ MONTADO

11. Visión general

11.1 Qué se construyó

La ERSO 008 **no añade un servicio nuevo**: es la **máquina de escritura** del VDR (ERSO 004) con todas sus puertas. Las piezas que implementan esta ERSO son:

Pieza	Qué hace
issueChallenge	Evalúa precondiciones y emite el desafío (criterios 1 y 2)
writeTx	Recorre las puertas de la escritura (criterios 3 y 4)
confirm / reconcile	Confirma y reconcilia (criterios 5 y 6)
versions / version / stateAt	Consulta de la traza (criterio 7)
Disparadores forbid_mutation	Hacen inalterables las versiones y la auditoría
Erso008LifecycleTest	8 pruebas automáticas

11.2 Las piezas en Docker

Contenedor	Para qué sirve en esta ERSO
nginx	Las dos puertas (lectura 8443, escritura 9443 con certificado cliente)
vdr	La aplicación: servidor público (8080) y de administración (8081)
postgres	Donde viven versiones, desafíos, operaciones y auditoría
tools (perfil tools)	Caja de herramientas para firmar
vdr-lento y confirm-lab (temporales, §22)	Segunda instancia del registro con confirmación inalcanzable, y un servicio que «reaparece»

11.3 El orden de las puertas en una escritura

Es el corazón de la ERSO. Las puertas, **en el orden real en que el código las evalúa**:

```
PUT /documents/{did} (If-Match, Idempotency-Key, challengeId, document, proof)
0   | If-Match presente y entero ≥ 0 ? ————— no → 428 / 400
1   | DID bien formado, de este dominio y en MI namespace ? — no → 400 / 422 / 403
2   | Documento válido (lista blanca; id; controladores) ? — no → 422 INVALID_DOCUMENT
3   | ¿Misma Idempotency-Key ya vista?
    |   | mismo contenido → devolver el resultado original (replayed:true) ← no gasta desafío
    |   | otro contenido → 422 IDEMPOTENCY_KEY_REUSED
4   | Consumir el desafío (atómico, un solo uso) ——— no → 403 CHALLENGE_INVALID
```

```

└─ ¿corresponde a este DID, cuenta, tipo y audiencia? – no → 403 CHALLENGE_MISMATCH
5 └─ Bloquear la fila del DID
    └─ ¿estado terminal? → 409 TERMINAL_STATE
    └─ ¿versión esperada = versión actual? — no → 412 VERSION_CONFLICT
    └─ Prueba de posesión: clave en authentication VIGENTE,
        firma válida y payload = {aud, challenge, did, docHash, purpose} – no → 403
INVALID_PROOF
6 └─ Guardar versión + operación (PENDING) + auditoría
7 └─ Confirmar leyendo la URL pública — coincide → CONFIRMED (200/201)
    no responde → PENDING (202)

```

💡 Observa tres detalles de diseño:

1. **La idempotencia se evalúa antes de gastar el desafío:** un reintento tras un corte de red recibe el resultado original sin necesitar otro desafío.
2. **El desafío se gasta aunque la operación falle más adelante** (por ejemplo, por versión errónea): es lo que significa «un solo uso».
3. **La prueba de posesión se verifica con la clave *vigente***, no con la del documento nuevo (excepto en `CREATE`, donde aún no hay historia).

12. El entorno para esta ERSO: el protocolo «a mano»

12.1 Por qué a mano

En las ERSO anteriores se usó `tools write`, que ejecuta las cuatro fases del protocolo de un tirón. Aquí **cada puerta es un criterio**, así que el protocolo se ejecuta **fase por fase** con `curl`, y la herramienta `tools` se usa solo para lo que no puede hacerse a mano: generar claves y **firmar**.

12.2 Los atajos (funciones de shell)

Atajo	Qué hace
<code>CURLM <entidad></code>	<code>curl</code> por la puerta de escritura (9443) presentando el certificado de esa entidad
<code>TOKEN <entidad> <secreto></code>	Obtiene el token de acceso (credenciales + certificado)
<code>API <entidad> <token> <método> <ruta></code>	Llama a una ruta administrativa con token
<code>DESAFIO <entidad> <token> <did> <propósito></code>	Pide un desafío
<code>NUEVO <did> <propósito></code>	Pide un desafío y guarda su identificador (<code>CHID</code>), su valor aleatorio (<code>NONCE</code>) y su audiencia (<code>AUD</code>)
<code>DOCHASH <archivo></code>	Calcula a mano la huella del documento (JSON canónico + SHA-256)
<code>PRUEBA <did> <propósito> <huella> <clave></code>	Construye el contenido a firmar {aud, challenge, did, docHash, purpose} y lo firma con la clave indicada
<code>PUTDOC <did> <versión> <idempotencia> <doc> <prueba></code>	Envía la escritura (PUT) con If-Match y Idempotency-Key

⚠️ **PRUEBA** recibe **qué clave** firma: eso permite probar con la clave vieja, la nueva o una ajena. Y recibe **la huella** y **el propósito**: eso permite firmar mal a propósito.

```
$ cd /home/geovani/Descargas/generic/bitacora/new_proyect/deploy
$ set -a; . ./env; set +a; export LOCAL_UID=$(id -u) LOCAL_GID=$(id -g)
$ D=$VDR_DOMAIN
$ W=./evidencias/work
$ PUB="--cacert certs/ca.crt --resolve $D:8443:127.0.0.1"
$ PSQL="docker compose exec -T postgres psql -U $POSTGRES_USER -d $POSTGRES_DB"
$ TOOLS() { docker compose --progress quiet --profile tools run --rm -T tools "$@"; }
$ ESCRIBIR() { local cid=$1 sec=$2 p12=$3 did=$4; shift 4; TOOLS write --admin-url "https://$D:8443"
--client-id "$cid" --secret "$sec" --p12 "/certs/$p12.p12" --p12-pass "$P12_PASS" --ca /certs/ca.crt
--did "$did" "$@"; }
$ CURLM() { local who=$1; shift; curl -s --cacert certs/ca.crt --resolve $D:9443:127.0.0.1 --cert
certs/$who.crt --key certs/$who.key "$@"; }
$ TOKEN() { CURLM $1 -X POST https://$D:9443/admin/v1/oauth/token -d grant_type=client_credentials -d
client_id=$1 -d client_secret=$2 | jq -r .access_token; }
$ API() { local who=$1 tk=$2 m=$3 p=$4; shift 4; CURLM $who -X $m "https://$D:9443/admin/v1$p" -H
"Authorization: Bearer $tk" "$@"; }
$ DESAFIO() { API $1 $2 POST /challenges -H "Content-Type: application/json" -d "
{'did\": \"$3\", \"purpose\": \"$4\"}"; }
$ DOCHASH() { jq -cS . $W/$1 | tr -d '\n' | sha256sum | awk '{print "sha256:$1}'; }
$ PRUEBA() { local p; p=$(jq -cn --arg a "$AUD" --arg c "$NONCE" --arg d "$1" --arg h "$3" --arg p
"$2" '{aud:$a,challenge:$c,did:$d,dochash:$h,purpose:$p}' | jq -cS .); TOOLS sign --key $4 --kid
"$1#key-1" --message "$p" | tail -1; }
$ PUTDOC() { jq -n --arg c "$CHID" --slurpfile d $W/$4 --arg p "$5"
'{"challengeId:$c,document:$d[0],proof:$p}' | API avance-issuer $TK PUT "/documents/$1" -w "\nHTTP %
{http_code}\n" -H "If-Match: $2" -H "Idempotency-Key: $3" -H "Content-Type: application/json" -d @-;
}
$ NUEVO() { local R; R=$(DESAFIO avance-issuer $TK $1 $2); CHID=$(echo "$R_" | jq -r .challengeId);
NONCE=$(echo "$R_" | jq -r .nonce); AUD=$(echo "$R_" | jq -r .audience); echo "$R_" | jq -c .; }
```

12.3 Comprobar que el registro está vivo

```
$ curl -s $PUB https://$D:8443/health
{"status": "UP", "vdr": "enabled"}
```

12.4 Un identificador de prueba único

Cada ejecución crea un DID `...:entidades:ciclo-008-<hora>`, para poder repetir sin choques (Avance tiene reservado `entidades/*`, ver informe de la ERSO 006, §12.5).

```
$ ID=ciclo-008-$(date +%s); DID=did:web:$D:entidades:$ID; echo "DID=$DID"
DID=did:web:civica-desarrollo.avance.org.co:entidades:ciclo-008-1790875523
```

13. Cómo funciona por dentro (lo mínimo para entender las pruebas)

13.1 Las tablas

```
$ cd /home/geovani/Descargas/generic/bitacora/new_proyect/deploy && set -a && . ./env && set +a
$ docker compose exec -T postgres psql -U $POSTGRES_USER -d $POSTGRES_DB -c "\dt"
      List of relations
Schema |      Name      | Type  | Owner
-----+-----+-----+-----
public | audit_log      | table | vdr
public | challenges     | table | vdr
public | did_document_versions | table | vdr
public | did_documents  | table | vdr
public | entity_accounts | table | vdr
public | namespaces     | table | vdr
```

```

public | operations | table | vdr
public | schema_migrations | table | vdr
public | wallet_audit | table | vdr
public | wallet_challenges | table | vdr
public | wallet_citizens | table | vdr
public | wallet_did_backups | table | vdr
public | wallet_did_publications | table | vdr
public | wallet_instances | table | vdr
public | wallet_recovery_tokens | table | vdr
(15 rows)

```

Las ocho primeras (de `audit_log` a `schema_migrations`) son del registro; las `wallet_*` son de la cartera (ERSo 001-003). Las que importan aquí:

Tabla	Guarda	¿Se puede modificar?
<code>did_documents</code>	El DID, su estado (<code>ACTIVE</code> / <code>DEACTIVATED</code>) y su versión actual	Sí (es el «puntero»)
<code>did_document_versions</code>	Todas las versiones con su hash y quién las hizo	No (solo-agregar)
<code>challenges</code>	Desafíos: tipo, audiencia, vencimiento, si se usaron	Sí (se marcan usados)
<code>operations</code>	Cada escritura: estado, versión, hash, URL	Sí (de <code>PENDING</code> a <code>CONFIRMED</code>)
<code>audit_log</code>	Cada evento	No (solo-agregar)

13.2 Qué impide modificar el pasado

```

$ docker compose exec -T postgres psql -U $POSTGRES_USER -d $POSTGRES_DB -c "select tgrelid::regclass
as tabla, tgname as disparador from pg_trigger where not tgisinternal order by 1"
      tabla      |      disparador
-----+-----
did_document_versions | versions_append_only
audit_log          | audit_append_only
wallet_audit       | wallet_audit_append_only
(3 rows)

```

Un **disparador** (*trigger*) es una regla que la base de datos ejecuta automáticamente. Estos tres rechazan cualquier `UPDATE` o `DELETE` sobre su tabla. Lo comprobaremos en §23.

13.3 El orden de las puertas en el código

```

$ grep -nE '^          // [0-9]\\)|^          // prueba de posesión' ../vdr-
service/src/main/kotlin/co/org/avance/ssi/vdr/RegistryService.kt | cut -c1-175
173:          // 1) el DID debe estar en un namespace de la entidad (autorización por namespace)
179:          // 2) documento: el perfil de publicación exige id igual al DID, sin claves privadas ni
datos civiles
193:          // 3) idempotencia: un reintento con la misma clave devuelve el resultado original SIN
gastar otro desafío
200:          // 4) el desafío se consume de forma atómica (un solo uso) y debe corresponder a ESTA
operación
210:          // 5) transacción de escritura bajo bloqueo de la fila del DID
227:          // prueba de posesión: firma sobre (desafío + hash) con una clave de `authentication`
vigente

```

Los números coinciden con el diagrama de §11.3.

13.4 Qué se firma exactamente

```
$ sed -n '/private fun expectedPayload/,/^    }/p' ../vdr-  
service/src/main/kotlin/co/org/avance/ssi/vdr/RegistryService.kt  
    private fun expectedPayload(ch: ChallengeRow, purpose: Purpose, docHash: String) =  
    buildJsonObject {  
        put("aud", JsonPrimitive(ch.audience)); put("challenge", JsonPrimitive(ch.nonce)); put("did",  
        JsonPrimitive(ch.did))  
        put("docHash", JsonPrimitive(docHash)); put("purpose", JsonPrimitive(purpose.name))  
    }
```

La prueba debe firmar **exactamente** este contenido —cinco campos—: la audiencia, el valor aleatorio del desafío, el DID, la huella del documento y el propósito. El registro lo reconstruye por su cuenta y lo compara con lo firmado; si no es idéntico, rechaza (`INVALID_PROOF`).

13.5 Plazos

```
$ grep -n 'challengeTtlSeconds' ../vdr-service/src/main/kotlin/co/org/avance/ssi/vdr/AppConfig.kt |  
head -2; grep -n 'confirmTimeoutMs' ../vdr-service/src/main/kotlin/co/org/avance/ssi/vdr/AppConfig.kt  
| head -1  
36:    val challengeTtlSeconds: Long = 300,  
67:        challengeTtlSeconds = env["CHALLENGE_TTL_SECONDS"]?.toLong() ?: 300,  
30:    val confirmTimeoutMs: Long = 3_000,
```

- El desafío dura **300 segundos** (5 min).
- La confirmación espera hasta **3 000 ms** (3 s) la respuesta de la URL pública.

13.6 Mapa de requisitos a implementación

Paso	Implementación	Dónde
1	<code>preconditionFailures</code>	<code>RegistryService.kt</code>
2	<code>POST /admin/v1/challenges</code>	<code>Server.kt</code> , <code>issueChallenge</code>
3	<code>DidDocumentValidator</code> (perfil <code>PUBLISHER</code>)	<code>did-core</code>
4	<code>verifyProof</code> (payload exacto)	<code>RegistryService.kt</code>
5	<code>PUT /documents/{did}</code> y <code>POST .../deactivate</code> bajo bloqueo de fila	<code>Server.kt</code> , <code>writeTx</code>
6	<code>confirm()</code>	<code>RegistryService.kt</code>
7	<code>PENDING</code> + <code>POST /operations/{id}/reconcile</code>	<code>RegistryService.kt</code>
8	<code>did_document_versions</code> + <code>/versions</code> , <code>/versions/{n}</code> , <code>/state?at=</code>	<code>Server.kt</code> , esquema SQL

PARTE IV — LABORATORIOS DE DEMOSTRACIÓN: LA PRUEBA MATERIAL DE CADA CRITERIO

14. Cómo funcionan los laboratorios

Cada laboratorio:  **predicción** → **comandos** (cada uno con su respuesta real y su explicación: qué hace, qué se busca, qué significa) →  **conclusión**.

 Es **una sola sesión continua** con un único DID de prueba que va **viviendo su ciclo de vida** a lo largo de los laboratorios: nace en el laboratorio B (criterio 2), cambia en el D (criterio 4), se da de baja en el G (criterio 7). Los laboratorios no son independientes. El script `scripts/lab-008-completo.sh` ejecuta todo seguido, incluida la limpieza.

15. Preparación común

La preparación (terminal, atajos del protocolo y DID de prueba) está en §12.2 a §12.4.

16. Mapa de los laboratorios

Lab	Criterio	Qué se prueba	Qué le pasa al DID de prueba	Sección
A	1	Las precondiciones bloquean el desafío	(aún no existe)	§17
B	2	Desafío de un solo uso, tipo y audiencia	Nace (v1)	§18
C	3	Documentos prohibidos se rechazan	(sobre otro DID)	§19
D	4	Versión esperada e idempotencia	Rota su clave (v2)	§20
E	5	Confirmación por lectura y hash	—	§21
F	6	Pendiente y reconciliar	(otro DID, con una segunda instancia)	§22
G	7	La traza reconstruye el estado	Se actualiza (v3) y se desactiva (v4)	§23
H	—	Pruebas automáticas	—	§24

17. CRITERIO 1 — Las precondiciones se cumplen; si alguna falla, no se emite desafío

Criterio. "Las precondiciones se cumplen; si alguna falla no se emite desafío; evidencia:

Qué se busca. Siete formas distintas de incumplir una precondition, y comprobar que **ninguna** produce desafío (el contador de desafíos no cambia) y que **todas** quedan en la auditoría con su motivo.

Predice antes de ejecutar

Voy a pedir desafíos en siete situaciones incorrectas. ¿Cuántos desafíos nuevos habrá en la base de datos después? ¿Y cuántas filas nuevas en la auditoría?

17.1 La situación inicial

```
$ echo "select client_id, enabled from entity_accounts order by 1" | $PSQL
  client_id  | enabled
-----+-----
avance-issuer | t
lab-operator  | t
vdr-admin     | t
wallet-backend | t
(4 rows)
$ TK=$(TOKEN avance-issuer $CLIENT_SECRET_AVANCE); TKL=$(TOKEN lab-operator $CLIENT_SECRET_LAB); echo
"desafíos guardados ANTES: $($PSQL -tAc "select count(*) from challenges")"
desafíos guardados ANTES: 54
```

Cuatro cuentas habilitadas. Se anota el **número de desafíos guardados antes** (en este ejemplo, el contador). Además se obtienen dos tokens: el de Avance (TK) y el del laboratorio (TKL).

17.2 Cinco preconditiones incumplidas

```
$ CASO() { printf '%-30s -> ' "$1"; shift; "$@" | jq -c '[".error, .details]'; }
$ CASO "1 espacio ajeno"          DESAFIO lab-operator      $TKL did:web:$D:entidades:avance UPDATE
$ CASO "2 ruta no reservada"      DESAFIO avance-issuer  $TK  did:web:$D:ciudadanos:ana CREATE
$ CASO "3 dominio ajeno"         DESAFIO avance-issuer  $TK  did:web:otro.dominio:entidades:x
CREATE
$ CASO "4 DID inválido"          DESAFIO avance-issuer  $TK  "did:web:$D:entidades:.." CREATE
$ CASO "5 certificado de otro"    DESAFIO lab-operator   $TK  $DID CREATE
1 espacio ajeno                  -> ["PRECONDITION_FAILED",["NAMESPACE_NOT_OWNED"]]
2 ruta no reservada              -> ["PRECONDITION_FAILED",["NAMESPACE_NOT_RESERVED"]]
3 dominio ajeno                  -> ["PRECONDITION_FAILED",["DID_DOMAIN_MISMATCH"]]
4 DID inválido                   -> ["PRECONDITION_FAILED",["DID_INVALID"]]
5 certificado de otro             -> ["PRECONDITION_FAILED",["WRITE_CHANNEL_NOT_RESTRICTED"]]
```

- **Qué hace.** CASO pide un desafío en una situación incorrecta y muestra el error y los details de la respuesta.
- **Qué significa.** Todas dan PRECONDITION_FAILED (HTTP 412), cada una con su motivo:

Caso	Qué se hizo mal	Motivo devuelto
1	El laboratorio pide un desafío sobre el DID de Avance	NAMESPACE_NOT_OWNED
2	Avance pide un DID en ciudadanos/..., ruta que nadie reservó	NAMESPACE_NOT_RESERVED
3	Avance pide un DID de otro dominio	DID_DOMAIN_MISMATCH

Caso	Qué se hizo mal	Motivo devuelto
4	Avance pide un DID mal formado (. .)	DID_INVALID
5	Token de Avance presentado con el certificado del laboratorio	WRITE_CHANNEL_NOT_RESTRICTED

17.3 Cuenta deshabilitada

```
$ echo "update entity_accounts set enabled=false where client_id='lab-operator'" | $PSQL
$ CASO "6 cuenta deshabilitada" DESAFIO lab-operator $TKL did:web:$D:lab:x CREATE
$ echo "update entity_accounts set enabled=true where client_id='lab-operator'" | $PSQL
UPDATE 1
6 cuenta deshabilitada -> ["PRECONDITION_FAILED",["ACCOUNT_MISSING_OR_DISABLED"]]
UPDATE 1
```

- **Qué hace.** Deshabilita la cuenta del laboratorio en la base de datos (UPDATE 1), intenta pedir un desafío con un token que ya tenía, y la vuelve a habilitar (UPDATE 1).
- **Qué significa.** ACCOUNT_MISSING_OR_DISABLED : **deshabilitar una cuenta tiene efecto inmediato**, incluso con un token todavía vigente.
- ⚠ La modificación directa de la base de datos es solo un recurso de laboratorio, y se deshace en el mismo paso.

17.4 Sin certificado de cliente

```
$ curl -s -o /dev/null -w "7 sin certificado de cliente -> HTTP %{http_code}\n" --cacert certs/ca.crt
--resolve $D:9443:127.0.0.1 -X POST https://$D:9443/admin/v1/challenges -H "Authorization: Bearer
$TK" -d '{}'
```

7 sin certificado de cliente -> HTTP 400

El portero (nginx) ni deja pasar: 400 . No hay siquiera petición para la aplicación.

17.5 Resultado: ningún desafío, siete rastros

```
$ echo "desafíos guardados DESPUÉS: $($PSQL -tAc "select count(*) from challenges")"
$ echo "select to_char(at,'HH24:MI:SS') hora, actor, action, detail from audit_log where
action='CHALLENGE_DENIED' order by id desc limit 6" | $PSQL
desafíos guardados DESPUÉS: 54
  hora   |   actor   |   action   |   detail
-----+-----+-----+-----
17:25:24 | lab-operator | CHALLENGE_DENIED | {"purpose": "CREATE", "failures":
["ACCOUNT_MISSING_OR_DISABLED"]}
17:25:24 | avance-issuer | CHALLENGE_DENIED | {"purpose": "CREATE", "failures":
["WRITE_CHANNEL_NOT_RESTRICTED"]}
17:25:24 | avance-issuer | CHALLENGE_DENIED | {"purpose": "CREATE", "failures": ["DID_INVALID"]}
17:25:24 | avance-issuer | CHALLENGE_DENIED | {"purpose": "CREATE", "failures":
["DID_DOMAIN_MISMATCH"]}
17:25:24 | avance-issuer | CHALLENGE_DENIED | {"purpose": "CREATE", "failures":
["NAMESPACE_NOT_RESERVED"]}
17:25:24 | lab-operator | CHALLENGE_DENIED | {"purpose": "UPDATE", "failures":
["NAMESPACE_NOT_OWNED"]}
(6 rows)
```

- **Primera línea.** El contador de desafíos **es el mismo** que antes: **cero desafíos nuevos**.
- **Tabla.** Seis filas CHALLENGE_DENIED (los seis casos que llegaron a la aplicación) con su motivo exacto, de más reciente a más antigua. El séptimo (sin certificado) quedó en nginx.

Conclusión — evidencia del criterio 1

- Siete incumplimientos de precondition, **cero** desafíos emitidos.
- Cada uno con un motivo distinto y trazable en la auditoría.
- Verificación de **cuenta** (casos 1, 2, 6) y de **canal** (casos 5 y 7), que es la evidencia que pide el criterio.

18. CRITERIO 2 — El desafío es de un solo uso, con tipo y audiencia específicos

Criterio. "El desafío es de un solo uso, con tipo y audiencia específicos de la operación DID; evidencia: registro de emisión del desafío."

Predice antes de ejecutar

(1) ¿Qué campos tendrá un desafío? (2) Si uso el mismo desafío dos veces, ¿qué pasa la segunda? (3) Si pido un desafío para UPDATE y lo uso para DEACTIVATE, ¿se acepta? (4) ¿Y si se vence?

18.1 Pedir un desafío de creación

```
$ NUEVO $DID CREATE
{"challengeId":"120ff195-3be6-421f-bf9e-e702407e2c2e","nonce":"Yy9DMM2MyenlfzVcnvbncY0U9dJPtaX3T47fuJ0xF7E","did":"did:web:civica-desarrollo.avance.org.co:entidades:ciclo-008-1790875523","purpose":"CREATE","audience":"vdr:civica-desarrollo.avance.org.co:did-operation","expiresAt":"2026-10-01T17:30:24.820344Z"}
```

- **Qué hace.** `NUEVO $DID CREATE` solicita al registro un desafío para **crear** el DID de prueba.
- **Qué significa.** La respuesta trae: `challengeId` (su identificador), `nonce` (el valor aleatorio, de 32 bytes), `did`, **purpose: CREATE** (el tipo), **audience** (`vdr:<dominio>:did-operation:` para qué sistema) y `expiresAt` (5 minutos).

18.2 El registro de emisión (evidencia pedida)

```
$ echo "select left(id::text,8) as id, purpose, audience, to_char(expires_at,'HH24:MI:SS') as vence,
used_at is not null as usado from challenges where id = '$CHID'" | $PSQL
  id      | purpose | audience | vence | usado
-----+-----+-----+-----+-----
 120ff195 | CREATE  | vdr:civica-desarrollo.avance.org.co:did-operation | 17:30:24 | f
(1 row)
$ echo "select action, detail from audit_log where did = '$DID' order by id" | $PSQL
  action | detail
-----+-----
CHALLENGE_DENIED | {"purpose": "CREATE", "failures": ["WRITE_CHANNEL_NOT_RESTRICTED"]}
CHALLENGE_ISSUED | {"purpose": "CREATE", "audience": "vdr:civica-desarrollo.avance.org.co:did-operation", "expiresAt": "2026-10-01T17:30:24.820344Z", "challengeId": "120ff195-3be6-421f-bf9e-e702407e2c2e"}
(2 rows)
```

- **Primera tabla** (challenges): el desafío guardado, con su tipo, audiencia, vencimiento y usado = f (falso: aún no se ha gastado).
- **Segunda tabla** (audit_log , filtrada por este DID): CHALLENGE_ISSUED con el identificador, el tipo, la audiencia y el vencimiento. Es **el registro de emisión** que pide el criterio. (Más arriba aparece un CHALLENGE_DENIED por WRITE_CHANNEL_NOT_RESTRICTED : es el caso 5 del laboratorio A, que usó este mismo DID.)

18.3 Preparar el documento y la huella

```
$ T00LS keygen --out c008-a.json; T00LS build-doc --did $DID --key c008-a.json --out c008-v1.json
>/dev/null; H=$(DOCHASH c008-v1.json); echo "huella del documento (calculada a mano): $H"
Clave P-256 generada. Multikey público: zDnaeUTYNnDx7dpMq3KtiJcr3PADTVoytTLJMWpCo1HPhfoPQ
huella del documento (calculada a mano):
sha256:28958a964f676f12969305efe2b56aae6275e310ad22ab5d1ecdf8d5fbc50da1
```

- Se genera la clave **A** y se construye el documento v1.
- La **huella la calculamos nosotros a mano** (DOCHASH: JSON canónico + SHA-256). Debe ser idéntica a la que calcule el registro; lo verificaremos en §18.5.

18.4 La prueba de posesión por dentro

```
$ J=$(PRUEBA $DID CREATE $H c008-a.json)
$ dec() { local s=$(echo "$1" | tr "_-" "/+"); while [ $(( ${#s} % 4 )) -ne 0 ]; do s="$s="; done;
echo "$s" | base64 -d 2>/dev/null; }
$ echo "contenido firmado:"; dec $(echo $J | cut -d. -f2) | jq .
contenido firmado:
{
  "aud": "vdr:civica-desarrollo.avance.org.co:did-operation",
  "challenge": "Yy9DMM2MyenlfzVcnvbnY0U9dJPtaX3T47fuJ0xF7E",
  "did": "did:web:civica-desarrollo.avance.org.co:entidades:ciclo-008-1790875523",
  "docHash": "sha256:28958a964f676f12969305efe2b56aae6275e310ad22ab5d1ecdf8d5fbc50da1",
  "purpose": "CREATE"
}
```

- **Qué hace.** PRUEBA arma el contenido con **cinco campos** (en orden alfabético y sin espacios) y lo firma con la clave A.
- **Qué significa.** Se ve exactamente **lo que se firma**:
 - aud y challenge: atan la firma a **este desafío**, que es de un solo uso.
 - did: a este DID.
 - docHash: a **este documento exacto**. Si alguien cambia el contenido, la huella ya no coincide.
 - purpose: a esta operación.

18.5 Primer (y único) uso del desafío

```
$ PUTDOC $DID 0 k1-$ID c008-v1.json "$J"
{"operationId":"47967937-1604-4a96-99f8-61583a3a3656","did":"did:web:civica-
desarrollo.avance.org.co:entidades:ciclo-008-
1790875523","purpose":"CREATE","status":"CONFIRMED","version":1,"hash":"sha256:28958a964f676f12969305
efe2b56aae6275e310ad22ab5d1ecdf8d5fbc50da1","publicUrl":"https://civica-
desarrollo.avance.org.co/entidades/ciclo-008-1790875523/did.json"}
HTTP 201
```

- **Qué hace.** PUTDOC envía la escritura: If-Match: 0 («no existe aún»), una clave de idempotencia, el documento y la prueba.
- **Qué significa.** 201 y CONFIRMED. El hash devuelto es **idéntico** al calculado a mano en §18.3: ambas partes coinciden sobre la huella.

```
$ echo "select left(id::text,8) as id, purpose, used_at is not null as usado from challenges where id = '$CHID'" | $PSQL
      id      | purpose | usado
-----+-----+-----
 120ff195 | CREATE  | t
(1 row)
```

El desafío aparece ahora con usado = t (verdadero): **se gastó**.

18.6 Segundo uso del mismo desafío

```
$ PUTDOC $DID 0 k2-$ID c008-v1.json "$J"
{"error":"CHALLENGE_INVALID","message":"Desafío inexistente, expirado o ya utilizado"}
HTTP 403
```

- **Qué hace.** Repite la escritura reutilizando el mismo desafío (con otra clave de idempotencia, para que no se tome como repetición).
- **Qué significa.** 403 CHALLENGE_INVALID: desafío «inexistente, expirado o ya utilizado». Es el **un solo uso**.

18.7 Un desafío de un tipo, usado para otro

```
$ NUEVO $DID UPDATE >/dev/null
$ H1=$(DOCHASH c008-v1.json); J=$(PRUEBA $DID DEACTIVATE $H1 c008-a.json)
$ jq -n --arg c "$CHID" --arg p "$J" '{challengeId:$c,proof:$p}' | API avance-issuer $TK POST
"/documents/$DID/deactivate" -w "\nHTTP %{http_code}\n" -H "If-Match: 1" -H "Idempotency-Key: k3-$ID"
-H "Content-Type: application/json" -d @-
{"error":"CHALLENGE_MISMATCH","message":"El desafío no corresponde a esta operación (tipo, DID,
audiencia o cuenta)"}
HTTP 403
```

- **Qué hace.** Pide un desafío para UPDATE, pero lo usa para una **desactivación** (con una prueba que declara DEACTIVATE).
- **Qué significa.** 403 CHALLENGE_MISMATCH: «no corresponde a esta operación (tipo, DID, audiencia o cuenta)». Un desafío es válido **solo** para el tipo que se pidió.

18.8 Un desafío vencido

```
$ T00LS keygen --out c008-b.json >/dev/null; T00LS build-doc --did $DID --key c008-b.json --out c008-v2.json >/dev/null
$ NUEVO $DID UPDATE >/dev/null; H2=$(DOCHASH c008-v2.json); J=$(PRUEBA $DID UPDATE $H2 c008-a.json)
$ echo "update challenges set expires_at = now() - interval '1 second' where id = '$CHID'" | $PSQL
$ PUTDOC $DID 1 k4-$ID c008-v2.json "$J"
UPDATE 1
{"error":"CHALLENGE_INVALID","message":"Desafío inexistente, expirado o ya utilizado"}
HTTP 403
```

- **Qué hace.** Genera la clave B (para la rotación del laboratorio D), pide un desafío UPDATE, **adelanta su vencimiento** en la base de datos (expires_at = ahora - 1 s; UPDATE 1) y

trata de usarlo.

- **Qué significa.** 403 CHALLENGE_INVALID. (Manipular el reloj en la base es un atajo de laboratorio para no esperar 5 minutos.)

18.9 Una audiencia ajena

```
$ NUEVO $DID UPDATE >/dev/null; AUDREAL=$AUD; AUD="vdr:otro.dominio:did-operation"
$ J=$(PRUEBA $DID UPDATE $H2 c008-a.json); AUD=$AUDREAL
$ PUTDOC $DID 1 k5-$ID c008-v2.json "$J"
{"error":"INVALID_PROOF","message":"Prueba de posesión inválida: el payload no corresponde al desafío/hash de la operación"}
HTTP 403
```

- **Qué hace.** Firma una prueba cuya audiencia es de **otro sistema** (vdr:otro.dominio:did-operation).
- **Qué significa.** 403 INVALID_PROOF: «el payload no corresponde al desafío/hash de la operación». Una prueba hecha para otro destinatario no sirve aquí.

🎯 Conclusión — evidencia del criterio 2

Propiedad	Evidencia	Resultado
Con tipo	purpose en el desafío; uso con otro tipo	CHALLENGE_MISMATCH
Con audiencia	audience en el desafío; prueba con otra	INVALID_PROOF
De un solo uso	usado = t; segundo uso	CHALLENGE_INVALID
Con vigencia	expiresAt; vencido	CHALLENGE_INVALID
Registro de emisión	CHALLENGE_ISSUED en la auditoría	✓

19. CRITERIO 3 — El documento no incluye claves privadas, y id y controladores corresponden al recurso autorizado

Criterio. "El documento no incluye claves privadas y el id y los controladores corresponden al recurso autorizado; evidencia: documento de la operación."

🔧 Predice antes de ejecutar

Si publico un documento con un id de otra entidad, ¿cuántas razones de rechazo espero?
¿Se guarda algo en la base de datos?

19.1 Preparar los documentos defectuosos

```
$ DIDN=did:web:$D:entidades:$ID-neg; TOOLS build-doc --did $DIDN --key c008-a.json --out c008-neg.json >/dev/null; echo "base de los intentos: $DIDN"
base de los intentos: did:web:civica-desarrollo.avance.org.co:entidades:ciclo-008-1790875523-neg
```

Se usa un DID distinto (...-neg) para no tocar el documento real del laboratorio.

19.2 Con clave privada

```
$ jq '.verificationMethod[0].privateKeyMultibase="z1234"' $W/c008-neg.json > $W/c008-n1.json; ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer $DIDN --purpose CREATE --expected 0 --key c008-a.json --doc c008-n1.json 2>&1 | tail -n +4
[4/4] escritura .... 422 If-Match=0 Idempotency-Key=c269f93e-e6ab-42cc-82e2-73dfec8cf372
{
  "error": "INVALID_DOCUMENT",
  "message": "El documento no cumple el perfil de publicación",
  "details": [
    "PRIVATE_KEY_MATERIAL: Campo prohibido con posible clave privada en
$.verificationMethod[0].privateKeyMultibase"
  ]
}
```

422 INVALID_DOCUMENT con PRIVATE_KEY_MATERIAL y la ruta exacta del campo prohibido.

19.3 Con un id que no es el del DID

```
$ jq '.id="did:web:$D:entidades:otra-entidad"' $W/c008-neg.json > $W/c008-n2.json; ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer $DIDN --purpose CREATE --expected 0 --key c008-a.json --doc c008-n2.json 2>&1 | tail -n +4
[4/4] escritura .... 422 If-Match=0 Idempotency-Key=c858822e-9b77-4a43-a1a1-4671093e734e
{
  "error": "INVALID_DOCUMENT",
  "message": "El documento no cumple el perfil de publicación",
  "details": [
    "ID_MISMATCH: El id 'did:web:civica-desarrollo.avance.org.co:entidades:otra-entidad' no coincide con el DID esperado 'did:web:civica-desarrollo.avance.org.co:entidades:ciclo-008-1790875523-neg'",
    "INVALID_VERIFICATION_METHOD: El id 'did:web:civica-desarrollo.avance.org.co:entidades:ciclo-008-1790875523-neg#key-1' debe ser una URL DID con fragmento del propio documento",
    "UNAUTHORIZED_CONTROLLER: did:web:civica-desarrollo.avance.org.co:entidades:ciclo-008-1790875523-neg#key-1: controlador no autorizado (did:web:civica-desarrollo.avance.org.co:entidades:ciclo-008-1790875523-neg)"
  ]
}
```

- **Qué significa.** Tres razones encadenadas: ID_MISMATCH (el id no es el DID pedido) y, como consecuencia, INVALID_VERIFICATION_METHOD (la clave ...-neg#key-1 ya no es «del propio documento») y UNAUTHORIZED_CONTROLLER (su controlador no es quien debería). Un solo cambio del id rompe tres reglas: el sistema revisa **la coherencia entera** del documento.

19.4 Con un controlador ajeno

```
$ jq '.controller="did:web:otra.entidad"' $W/c008-neg.json > $W/c008-n3.json; ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer $DIDN --purpose CREATE --expected 0 --key c008-a.json --doc c008-n3.json 2>&1 | tail -n +4
[4/4] escritura .... 422 If-Match=0 Idempotency-Key=c5aa7430-7dd3-45d8-984c-577b5248a87e
{
  "error": "INVALID_DOCUMENT",
  "message": "El documento no cumple el perfil de publicación",
  "details": [
    "UNAUTHORIZED_CONTROLLER: Controlador no autorizado: did:web:otra.entidad"
  ]
}
```

UNAUTHORIZED_CONTROLLER: solo el propio DID puede ser el controlador.

19.5 Con datos civiles

```
$ jq '.credentialSubject={"name":"Ana Pérez"}' $W/c008-neg.json > $W/c008-n4.json; ESCRIBIR avance-  
issuer $CLIENT_SECRET_AVANCE avance-issuer $DIDN --purpose CREATE --expected 0 --key c008-a.json --  
doc c008-n4.json 2>&1 | tail -n +4  
[4/4] escritura .... 422 If-Match=0 Idempotency-Key=63cbfcc2-feb1-4a85-82ea-0676f107b72c  
{  
  "error": "INVALID_DOCUMENT",  
  "message": "El documento no cumple el perfil de publicación",  
  "details": [  
    "CIVIL_DATA: Campo de datos civiles no permitido en $.credentialSubject",  
    "CIVIL_DATA: Campo de datos civiles no permitido en $.credentialSubject.name",  
    "UNKNOWN_PROPERTY: Propiedad no permitida en el perfil de publicación: 'credentialSubject'"  
  ]  
}
```

Tres razones: el campo `credentialSubject`, su contenido `name` y la propiedad desconocida (fuera de la lista blanca).

19.6 No quedó nada

```
$ echo "documentos registrados con ese identificador: $($PSQL -tAc "select count(*) from  
did_documents where did = '$DIDN'")"  
documentos registrados con ese identificador: 0
```

Cero documentos registrados: un rechazo es total.

Conclusión — evidencia del criterio 3

Defecto	Respuesta
Clave privada	PRIVATE_KEY_MATERIAL
id ajeno	ID_MISMATCH (+ coherencia)
Controlador ajeno	UNAUTHORIZED_CONTROLLER
Datos civiles	CIVIL_DATA + UNKNOWN_PROPERTY
Efecto	422 y 0 documentos guardados

20. CRITERIO 4 — La escritura exige la versión esperada y devuelve versión, hash y URL pública

Criterio. "La escritura exige la versión esperada y devuelve versión, hash y URL pública; evidencia: respuesta del registro."

Predice antes de ejecutar

(1) Si escribo sin `If-Match`, ¿qué código? (2) El DID está en la versión 1; escribo diciendo que espero la 5. ¿Qué responde el registro? (3) Si repito exactamente la misma petición con la misma clave de idempotencia, ¿se crea la versión 3?

20.1 Sin versión esperada

```
$ API avance-issuer $TK PUT "/documents/$DID" -w "\nHTTP %{http_code}\n" -H "Idempotency-Key: z-$ID"
-H "Content-Type: application/json" -d '{}'
```

```
{"error":"IF_MATCH_REQUIRED","message":"La escritura exige If-Match con la versión esperada (0 para
crear)"}
HTTP 428
```

428 IF_MATCH_REQUIRED: sin versión esperada, **ni se mira** el resto.

20.2 Con una versión equivocada

```
$ NUEVO $DID UPDATE >/dev/null; J=$(PRUEBA $DID UPDATE $H2 c008-a.json)
$ PUTDOC $DID 5 k6-$ID c008-v2.json "$J"
```

```
{"error":"VERSION_CONFLICT","message":"Versión esperada 5, versión actual 1","details":
["currentVersion=1"]}
```

```
HTTP 412
```

- **Qué hace.** Con un desafío y una prueba perfectos, declara esperar la versión 5.
- **Qué significa.** 412 VERSION_CONFLICT: «Versión esperada 5, versión actual 1», con `currentVersion=1` para que el cliente sepa dónde está. Es el control de concurrencia: **alguien más pudo haber cambiado el documento.**

20.3 Con la versión correcta: rotación de clave

```
$ NUEVO $DID UPDATE >/dev/null; J=$(PRUEBA $DID UPDATE $H2 c008-a.json)
$ OUT=$(PUTDOC $DID 1 k7-$ID c008-v2.json "$J"); echo "$OUT"; OP2=$(echo "$OUT" | head -1 | jq -r
.operationId); CH2=$CHID; J2=$J
```

```
{"operationId":"daeb9264-6779-4ee4-a17a-224622301b6c","did":"did:web:civica-
desarrollo.avance.org.co:entidades:ciclo-008-
1790875523","purpose":"UPDATE","status":"CONFIRMED","version":2,"hash":"sha256:99fbc116a55c2d9da67b07
996998cc8e2c0aa807c30c800cfd08ae392c7b0f4f","publicUrl":"https://civica-
desarrollo.avance.org.co/entidades/ciclo-008-1790875523/did.json"}
```

```
HTTP 200
```

- **Qué hace.** If-Match: 1, documento v2 (con la clave **B**), prueba firmada con la clave **A**, la vigente.
- **Qué significa.** 200, UPDATE, CONFIRMED, **versión 2, hash y URL pública:** los tres datos que pide el criterio. La prueba la firmó la clave *vieja* (la que gobernaba el DID); el documento nuevo y a lleva la *nueva*: eso es una **rotación**.

20.4 Repetir la misma petición

```
$ CHID=$CH2; PUTDOC $DID 1 k7-$ID c008-v2.json "$J2"
```

```
{"operationId":"daeb9264-6779-4ee4-a17a-224622301b6c","did":"did:web:civica-
desarrollo.avance.org.co:entidades:ciclo-008-
1790875523","purpose":"UPDATE","status":"CONFIRMED","version":2,"hash":"sha256:99fbc116a55c2d9da67b07
996998cc8e2c0aa807c30c800cfd08ae392c7b0f4f","publicUrl":"https://civica-
desarrollo.avance.org.co/entidades/ciclo-008-1790875523/did.json","replayed":true}
```

```
HTTP 200
```

- **Qué significa.** Mismo `operationId`, misma versión 2 y la marca `"replayed": true`: el registro reconoció la clave de idempotencia y devolvió el resultado original **sin crear otra versión y sin gastar otro desafío.**

20.5 La misma clave con otro contenido

```
$ TTOOLS build-doc --did $DID --key c008-b.json --service-url "https://$D/issuer" --out c008-v3.json >/dev/null; H3=$(DOCHASH c008-v3.json)
$ NUEVO $DID UPDATE >/dev/null; J=$(PRUEBA $DID UPDATE $H3 c008-b.json)
$ PUTDOC $DID 2 k7-$ID c008-v3.json "$J"
{"error":"IDEMPOTENCY_KEY_REUSED","message":"La clave de idempotencia ya se usó con otra solicitud"}
HTTP 422
```

422 `IDEMPOTENCY_KEY_REUSED`: la misma clave identifica una intención; no puede usarse para otra cosa distinta.

20.6 Comprobar que no hay versiones de más

```
$ echo "select version, operation, actor from did_document_versions where did = '$DID' order by version" | $PSQL
version | operation | actor
-----+-----+-----
1 | CREATE | avance-issuer
2 | UPDATE | avance-issuer
(2 rows)
```

Solo **dos** versiones (1 y 2): ni el reintento ni el rechazo crearon otras.

20.7 La clave vieja ya no sirve

```
$ NUEVO $DID UPDATE >/dev/null; J=$(PRUEBA $DID UPDATE $H3 c008-a.json)
$ PUTDOC $DID 2 k8-$ID c008-v3.json "$J"
{"error":"INVALID_PROOF","message":"Prueba de posesión inválida: firma no válida"}
HTTP 403
```

- **Qué hace.** Intenta una actualización con la prueba firmada por la clave **A**, que fue sustituida.
- **Qué significa.** 403 `INVALID_PROOF` («firma no válida»): el registro verifica con la clave **vigente** (la B). Una clave rotada pierde poder de escritura de inmediato.

Conclusión — evidencia del criterio 4

Situación	Respuesta
Sin <code>If-Match</code>	428
Versión errónea	412 <code>VERSION_CONFLICT</code> (+ <code>currentVersion</code>)
Versión correcta	200/201 con versión, hash y URL
Reintento idéntico	misma operación, <code>replayed: true</code>
Misma clave, otro contenido	422 <code>IDEMPOTENCY_KEY_REUSED</code>
Efecto neto	2 versiones

21. CRITERIO 5 — La publicación se confirma por lectura de la URL y comparación de hash

Criterio. "La publicación se confirma por lectura de la URL y comparación de hash; evidencia: comparación de contenido y hash."

Predice antes de ejecutar

¿Qué tres hashes distintos pueden compararse para la actualización v2 y qué relación debe haber entre ellos?

21.1 La evidencia de la operación

```
$ echo "operación de la actualización (v2): $0P2"; API avance-issuer $TK GET "/operations/$0P2" | jq .
operación de la actualización (v2): daeb9264-6779-4ee4-a17a-224622301b6c
{
  "id": "daeb9264-6779-4ee4-a17a-224622301b6c",
  "did": "did:web:civica-desarrollo.avance.org.co:entidades:ciclo-008-1790875523",
  "purpose": "UPDATE",
  "clientId": "avance-issuer",
  "status": "CONFIRMED",
  "version": 2,
  "hash": "sha256:99fbc116a55c2d9da67b07996998cc8e2c0aa807c30c800cfd08ae392c7b0f4f",
  "publicUrl": "https://civica-desarrollo.avance.org.co/entidades/ciclo-008-1790875523/did.json",
  "idempotencyKey": "k7-ciclo-008-1790875523",
  "requestHash": "sha256:d79f5b096ebdbf3100052b82224abb2c8ebaf0db7afcc1826fcf6d9ffd2adcd4",
  "createdAt": "2026-10-01T17:25:47.601380Z",
  "confirmedAt": "2026-10-01T17:25:47.616720Z"
}
```

- **Estado** **CONFIRMED** con `confirmedAt` unos milisegundos después de `createdAt`: el registro, tras escribir, **leyó su propia URL pública** y comparó.

21.2 Comparar con lo que se sirve hoy

```
$ echo "hash de la evidencia      : $(API avance-issuer $TK GET /operations/$0P2 | jq -r .hash)"
$ echo "hash del documento público: $(curl -s $PUB https://$D:8443/entidades/$ID/did.json | sha256sum | awk '{print "sha256:$1}')"
$ curl -s -D - -o /dev/null $PUB https://$D:8443/entidades/$ID/did.json | grep -iE "^(HTTP|etag|x-content-hash)"
hash de la evidencia      : sha256:99fbc116a55c2d9da67b07996998cc8e2c0aa807c30c800cfd08ae392c7b0f4f
hash del documento público: sha256:99fbc116a55c2d9da67b07996998cc8e2c0aa807c30c800cfd08ae392c7b0f4f
HTTP/1.1 200 OK
ETag: "2"
X-Content-Hash: sha256:99fbc116a55c2d9da67b07996998cc8e2c0aa807c30c800cfd08ae392c7b0f4f
```

- **Qué hace.** Muestra el hash de la evidencia, calcula el hash del documento **tal como lo entrega ahora** la URL pública, y muestra los encabezados `ETag` y `X-Content-Hash`.
- **Qué significa.** Los hashes son **idénticos**; el `ETag "2"` corresponde a la versión 2. La publicación se confirma por **contenido y hash**.

21.3 El contenido es el de la nueva clave

```
$ curl -s $PUB https://$D:8443/entidades/$ID/did.json | jq -c
'.verificationMethod[0].publicKeyMultibase'; echo "(clave b de la v2: $(jq -r .publicKeyMultibase
$W/c008-b.json))"
"zDnaeqeTQ17rAhNcbVSEk4SMqfnubeWmuJ5nbm11Fi6vETWvv"
(clave b de la v2: zDnaeqeTQ17rAhNcbVSEk4SMqfnubeWmuJ5nbm11Fi6vETWvv)
```

La clave publicada es la **B** (la nueva): la rotación es visible para cualquiera que lea la URL.

Conclusión — evidencia del criterio 5

CONFIRMED + hash de la evidencia = hash del documento servido = hash calculado a mano (§18.3 para v1, d3 para v2).

22. CRITERIO 6 — Si no hay respuesta en plazo, la operación queda pendiente y no se da por publicada

Criterio. *"Si no hay respuesta en plazo, la operación queda pendiente y no se da por publicada; evidencia: estado de la operación."*

Qué se busca. Provocar que la confirmación **no pueda leer** la URL y observar el estado PENDING ; luego restablecer y **reconciliar**.

Predice antes de ejecutar

Si la lectura de confirmación falla, ¿la escritura se deshace? ¿Qué código HTTP responde el registro? ¿Qué hace reconcile mientras el servicio siga inaccesible?

22.1 Cómo se provoca (y por qué hace falta una segunda instancia)

El registro real confirma leyendo **su propia dirección pública**, que funciona. Para que falle sin romper el sistema, se levanta una **segunda instancia temporal** (vdr-lento) conectada a la **misma base de datos** pero con la dirección de confirmación apuntando a un nombre que **no existe** (confirm.lab). Se prepara primero el certificado y el servicio que luego «reaparecerá»:

```
$ CD=$W/confirm008; rm -rf $CD; mkdir -p $CD
> openssl req -new -newkey ec -pkeyopt ec_paramgen_curve:prime256v1 -nodes -keyout $CD/c.key -subj
"/CN=confirm.lab" -out $CD/c.csr 2>/dev/null
> printf "subjectAltName=DNS:confirm.lab\n" > $CD/san.cnf
> openssl x509 -req -in $CD/c.csr -CA certs/ca.crt -CAkey certs/ca.key -set_serial 0x1008 -days 30 -
extfile $CD/san.cnf -out $CD/c.crt 2>/dev/null
> cat > $CD/c.conf <<'EOF'
> server {
>     listen 443 ssl;
>     server_name confirm.lab;
>     ssl_certificate /etc/nginx/c.crt;
>     ssl_certificate_key /etc/nginx/c.key;
>     location / { proxy_pass http://vdr:8080; }
> }
> EOF
> ls $CD
```

```
c.conf
c.crt
c.csr
c.key
san.cnf
```

22.2 Levantar la instancia con la confirmación rota

```
$ docker rm -f vdr-lento >/dev/null 2>&1
$ docker run -d --name vdr-lento --network vdr-ssi_default -p 127.0.0.1:18081:8081 \
$ -e VDR_ENABLED=true -e VDR_DOMAIN=$D -e PUBLIC_BASE_URL=https://$D \
$ -e CONFIRM_BASE_URL=https://confirm.lab -e CONFIRM_CA_PEM=/certs/ca.crt -e
CONFIRM_TIMEOUT_MS=2000 \
$ -e DB_URL=jdbc:postgresql://postgres:5432/$POSTGRES_DB -e DB_USER=$POSTGRES_USER -e
DB_PASSWORD=$POSTGRES_PASSWORD \
$ -e VDR_JWT_SECRET=$VDR_JWT_SECRET -e VDR_CLIENTS="$VDR_CLIENTS" -e VDR_REQUIRE_MTLS=false \
$ -v "$PWD/certs/ca.crt:/certs/ca.crt:ro" vdr-ssi/vdr-service:local >/dev/null
$ until curl -s -o /dev/null http://127.0.0.1:18081/admin/v1/oauth/token; do sleep 1; done
$ docker ps --filter name=vdr-lento --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES          STATUS          PORTS
vdr-lento      Up 1 second     127.0.0.1:18081->8081/tcp
```

- **Qué hace.** `docker run` lanza el mismo programa del registro con tres cambios: `CONFIRM_BASE_URL=https://confirm.lab` (destino inalcanzable), `CONFIRM_TIMEOUT_MS=2000` (plazo de 2 s), `VDR_REQUIRE_MTLS=false` (para atenderla directamente sin pasar por nginx). Solo se publica en `127.0.0.1:18081` (no es accesible desde fuera del equipo). El bucle espera a que responda.
- **Qué significa.** Aparece `Up` y escuchando en `127.0.0.1:18081`.

22.3 Publicar con la confirmación rota

```
$ API2() { local m=$1 p=$2; shift 2; curl -s -X $m "http://127.0.0.1:18081/admin/v1$p" -H
"Authorization: Bearer $TK2" "$@"; }
$ TK2=$(curl -s http://127.0.0.1:18081/admin/v1/oauth/token -d grant_type=client_credentials -d
client_id=avance-issuer -d client_secret=$CLIENT_SECRET_AVANCE | jq -r .access_token)
$ DIDP=did:web:$D:entidades:pend-$ID
$ TOOLS keygen --out c008-p.json >/dev/null; TOOLS build-doc --did $DIDP --key c008-p.json --out
c008-pdoc.json >/dev/null; HP=$(DOCHASH c008-pdoc.json)
$ R_=$(API2 POST /challenges -H "Content-Type: application/json" -d "
{"did\":\"$DIDP\", \"purpose\":\"CREATE\"}")
$ CHID=$(echo "$R_" | jq -r .challengeId); NONCE=$(echo "$R_" | jq -r .nonce); AUD=$(echo "$R_" | jq
-r .audience)
$ JP=$(PRUEBA $DIDP CREATE $HP c008-p.json)
$ jq -n --arg c "$CHID" --slurpfile d $W/c008-pdoc.json --arg p "$JP"
'{"challengeId:$c,document:$d[0],proof:$p}' | API2 PUT "/documents/$DIDP" -w "\nHTTP %{http_code}\n" -
H "If-Match: 0" -H "Idempotency-Key: kp-$ID" -H "Content-Type: application/json" -d @-
{"operationId":"63e0104b-e9f8-4bdd-914b-3c229e3d885f","did":"did:web:civica-
desarrollo.avance.org.co:entidades:pend-ciclo-008-
1790875523","purpose":"CREATE","status":"PENDING","version":1,"hash":"sha256:cd396a844b46438b1e24c914
e6babaa6b892e7474315ecdd7752cc35cd5f2e4d","publicUrl":"https://civica-
desarrollo.avance.org.co/entidades/pend-ciclo-008-1790875523/did.json"}
HTTP 202
```

- **Qué hace.** Completa el protocolo (token → desafío → prueba → PUT) contra la instancia `vdr-lento`, para un DID nuevo (pend-...).
- **Qué significa.** `202` con `"status":"PENDING"`: «aceptado, pero no confirmado». La respuesta sí trae versión, hash y URL.

22.4 El estado y el rastro

```
$ OPP=$(echo "select id from operations where did='$DIDP'" | $PSQL -tA)
$ echo "select purpose, status, version, confirmed_at is not null as confirmada from operations where
id='$OPP'" | $PSQL
$ echo "select action, detail from audit_log where did='$DIDP' order by id" | $PSQL
  purpose | status | version | confirmada
-----+-----+-----+-----
  CREATE | PENDING | 1 | f
(1 row)

      action      |
detail
-----+-----
CHALLENGE_ISSUED | {"purpose": "CREATE", "audience": "vdr:civica-desarrollo.avance.org.co:did-
operation", "expiresAt": "2026-10-01T17:30:55.897654Z", "challengeId": "3e477646-5d91-49c2-8a18-
86d0a4edcb00"}
WRITE_CREATE      | {"hash":
"sha256:cd396a844b46438b1e24c914e6babaa6b892e7474315ecdd7752cc35cd5f2e4d", "operationId": "63e0104b-
e9f8-4bdd-914b-3c229e3d885f", "idempotencyKey": "kp-ciclo-008-1790875523", "expectedVersion": 0}
PUBLICATION_PENDING | {"url": "https://confirm.lab/entidades/pend-ciclo-008-1790875523/did.json",
"detail": "error de lectura: UnresolvedAddressException", "operationId": "63e0104b-e9f8-4bdd-914b-
3c229e3d885f"}
(3 rows)
```

- **Tabla de la operación:** PENDING, versión 1, confirmada = f (falso).
- **Auditoría:** WRITE_CREATE (se guardó) y PUBLICATION_PENDING con el motivo: UnresolvedAddressException (el nombre confirm.lab no existe).
- **Lectura clave:** la escritura **no se deshizo** (la versión 1 está guardada), pero la operación **no se da por publicada**. Quien dependa de esta publicación debe esperar la confirmación.

22.5 Reconciliar mientras sigue roto

```
$ API2 POST "/operations/$OPP/reconcile" | jq -c '{status, version, hash: .hash[0:20]}'
{"status":"PENDING","version":1,"hash":"sha256:cd396a844b464"}

```

reconcile relee la URL; como sigue inaccesible, el estado **continúa** PENDING.

22.6 Hacer reaparecer el servicio

```
$ docker rm -f confirm-lab >/dev/null 2>&1
$ docker run -d --name confirm-lab --network vdr-ssi_default --network-alias confirm.lab -v
"$PWD/$CD/c.conf:/etc/nginx/conf.d/default.conf:ro" -v "$PWD/$CD/c.crt:/etc/nginx/c.crt:ro" -v
"$PWD/$CD/c.key:/etc/nginx/c.key:ro" nginx:1.27-alpine >/dev/null; sleep 2
$ docker ps --filter name=confirm-lab --format "table {{.Names}}\t{{.Status}}"
NAMES      STATUS
confirm-lab Up 2 seconds

```

Se levanta confirm-lab: un nginx con el nombre de red confirm.lab, el certificado firmado por nuestra CA, que **redirige las lecturas al registro real** (el que comparte la base de datos y, por tanto, sirve el documento recién guardado).

22.7 Reconciliar con el servicio de vuelta

```
$ sleep 15 # la JVM recuerda ~10 s que el nombre no existía
$ API2 POST "/operations/$OPP/reconcile" | jq -c '{status, version, hash: .hash[0:20]}'
$ echo "select action, detail from audit_log where did='$DIDP' order by id" | $PSQL
{"status":"CONFIRMED","version":1,"hash":"sha256:cd396a844b464"}
  action      |
detail
-----+-----
CHALLENGE_ISSUED | {"purpose": "CREATE", "audience": "vdr:civica-desarrollo.avance.org.co:did-operation", "expiresAt": "2026-10-01T17:30:55.897654Z", "challengeId": "3e477646-5d91-49c2-8a18-86d0a4edcb00"}
WRITE_CREATE      | {"hash":
"sha256:cd396a844b46438b1e24c914e6babaa6b892e7474315ecdd7752cc35cd5f2e4d", "operationId": "63e0104b-e9f8-4bdd-914b-3c229e3d885f", "idempotencyKey": "kp-ciclo-008-1790875523", "expectedVersion": 0}
PUBLICATION_PENDING | {"url": "https://confirm.lab/entidades/pend-ciclo-008-1790875523/did.json",
"detail": "error de lectura: UnresolvedAddressException", "operationId": "63e0104b-e9f8-4bdd-914b-3c229e3d885f"}
PUBLICATION_PENDING | {"url": "https://confirm.lab/entidades/pend-ciclo-008-1790875523/did.json",
"detail": "error de lectura: UnresolvedAddressException", "operationId": "63e0104b-e9f8-4bdd-914b-3c229e3d885f"}
PUBLICATION_CONFIRMED | {"url": "https://confirm.lab/entidades/pend-ciclo-008-1790875523/did.json",
"detail": "hash coincide", "operationId": "63e0104b-e9f8-4bdd-914b-3c229e3d885f"}
(5 rows)
```

- **Qué hace.** Espera 15 segundos (la máquina virtual de Java **recuerda unos 10 segundos** que un nombre no existía; sin esa espera, el primer intento seguiría fallando por la memoria de nombres, no por el servicio). Luego invoca `reconcile`.
- **Qué significa.**
 - La primera línea: el estado ahora es `CONFIRMED`.
 - En la auditoría: dos `PUBLICATION_PENDING` (el intento inicial y la reconciliación fallida), y por fin `PUBLICATION_CONFIRMED` con «hash coincide».

Conclusión — evidencia del criterio 6

Momento	Estado	HTTP	Evidencia
Servicio de confirmación caído	PENDING	202	PUBLICATION_PENDING
reconcile con servicio aún caído	PENDING	—	otro PUBLICATION_PENDING
reconcile con servicio restablecido	CONFIRMED	—	PUBLICATION_CONFIRMED («hash coincide»)

23. CRITERIO 7 — La traza permite reconstruir el estado en cada momento

Criterio. "La traza permite reconstruir el estado en cada momento; evidencia: historial de versiones."

Predice antes de ejecutar

Después de crear, rotar, añadir un servicio y desactivar, ¿cuántas versiones habrá? ¿Qué hash tendrá la de la desactivación? ¿Qué debe responder el registro si pregunto por un instante anterior a la creación?

23.1 Dos operaciones más: actualizar y desactivar

El DID de prueba está en la versión 2 (tras la rotación). Se añade un servicio (v3), firmando con la clave **B** (la vigente), y luego se desactiva (v4):

```
$ ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer $DID --purpose UPDATE --expected 2 --key c008-b.json --doc c008-v3.json 2>&1 | grep -E "^[4/4\]|\"status\"|\"version\""
[4/4] escritura .... 200 If-Match=2 Idempotency-Key=35954c7c-5010-48da-bd06-6f385c4c16db
  "status": "CONFIRMED",
  "version": 3,
$ ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer $DID --purpose DEACTIVATE --expected 3 -
-key c008-b.json 2>&1 | grep -E "^[4/4\]|\"status\"|\"purpose\"|\"version\""
[4/4] escritura .... 200 If-Match=3 Idempotency-Key=e5b77d27-ae63-4dea-b065-2db24c07c7f8
  "purpose": "DEACTIVATE",
  "status": "CONFIRMED",
  "version": 4,
```

- UPDATE con `--expected 2` → versión 3.
- DEACTIVATE con `--expected 3` → versión 4. La desactivación **no lleva documento nuevo**: la prueba firma el hash de la última versión.

23.2 El historial

```
$ TK=$(TOKEN avance-issuer $CLIENT_SECRET_AVANCE); API avance-issuer $TK GET
"/documents/$DID/versions" | jq -r '.[ ] | "v\(.version) \(.operation) \(.createdAt[11:23]) \
(.actor) \(.hash[0:26])..."
v1 CREATE 17:25:28.879 avance-issuer sha256:28958a964f676f12969...
v2 UPDATE 17:25:47.601 avance-issuer sha256:99fbc116a55c2d9da67...
v3 UPDATE 17:26:16.820 avance-issuer sha256:2ddd6d4f2cc2d33f78...
v4 DEACTIVATE 17:26:18.786 avance-issuer sha256:2ddd6d4f2cc2d33f78...
```

✓ **Cómo leerlo:** cuatro versiones con su operación, su instante, quién las hizo y su huella:

v	Operación	Qué cambió
1	CREATE	Nace con la clave A
2	UPDATE	Rotación a la clave B
3	UPDATE	Se añade el servicio de emisión
4	DEACTIVATE	Baja; conserva el hash de la v3 (el último documento)

23.3 Una versión concreta

```
$ API avance-issuer $TK GET "/documents/$DID/versions/2" | jq '{version, operation, hash,
claveEnElDocumento: (.document | fromjson | .verificationMethod[0].publicKeyMultibase)}'
{
  "version": 2,
  "operation": "UPDATE",
  "hash": "sha256:99fbc116a55c2d9da67b07996998cc8e2c0aa807c30c800cfd08ae392c7b0f4f",
```

```
"claveEnElDocumento": "zDnaeqeTQ17rAhNcbVSEk4SMqfnubEWmuJ5nbm1lFi6vETWvv"
}
```

Se puede pedir cualquier versión y obtener su contenido exacto: aquí, la clave de la v2.

23.4 El estado en cada instante

```
$ API avance-issuer $TK GET "/documents/$DID/versions" > $W/c008-versiones.json
$ for n in 1 2 3 4; do T=$(jq -r ".[${n-1}].createdAt" $W/c008-versiones.json); printf "en el
instante de v$n (%s) -> " "${T:11:12}"; API avance-issuer $TK GET "/documents/$DID/state?at=$T" | jq
-c '{status, version}'; done
en el instante de v1 (17:25:28.879) -> {"status":"ACTIVE","version":1}
en el instante de v2 (17:25:47.601) -> {"status":"ACTIVE","version":2}
en el instante de v3 (17:26:16.820) -> {"status":"ACTIVE","version":3}
en el instante de v4 (17:26:18.786) -> {"status":"DEACTIVATED","version":4}
```

- **Qué hace.** Para cada versión, toma su instante y pregunta al registro «¿cuál era el estado en ese momento?».
- **Qué significa.** En el instante de v1: ACTIVE, versión 1; v2: ACTIVE, 2; v3: ACTIVE, 3; v4: DEACTIVATED, 4. La traza **reconstruye** el estado en cada punto.

23.5 Antes de existir

```
$ API avance-issuer $TK GET "/documents/$DID/state?at=2000-01-01T00:00:00Z" -w "\nHTTP %
{http_code}\n"
{"error":"NOT_FOUND","message":"El DID no existía en 2000-01-01T00:00:00Z"}
HTTP 404
```

404: «El DID no existía en 2000-01-01...». También se reconstruye lo que **no** había.

23.6 Lo que ve el público

```
$ curl -s -o /dev/null -w "URL pública del DID desactivado -> HTTP %{http_code}\n" $PUB
https://$D:8443/entidades/$ID/did.json
URL pública del DID desactivado -> HTTP 410
```

410 Gone: el documento desactivado no se sirve, y el consumidor lo interpreta como «desactivado» (ERSo 007).

23.7 El estado terminal

```
$ ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer $DID --purpose CREATE --expected 0 --key
c008-a.json --doc c008-v1.json 2>&1
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío FALLÓ: HTTP 409 {"error":"TERMINAL_STATE","message":"El historial está desactivado; se
requiere una nueva instancia (otro namespace)"}
```

409 TERMINAL_STATE: «El historial está desactivado; se requiere una nueva instancia (otro namespace)». Ya **no se puede ni pedir un desafío** para ese DID. Es la regla transversal de la ERSo.

23.8 El pasado no se puede reescribir

```
$ echo "update did_document_versions set hash='x' where did='$DID'" | $PSQL 2>&1 | head -2
$ echo "delete from audit_log where did='$DID'" | $PSQL 2>&1 | head -2
ERROR:  tabla append-only: UPDATE no permitido sobre did_document_versions
CONTEXT:  PL/pgSQL function forbid_mutation() line 3 at RAISE
ERROR:  tabla append-only: DELETE no permitido sobre audit_log
CONTEXT:  PL/pgSQL function forbid_mutation() line 3 at RAISE
```

- **Qué hace.** Intenta, directamente en la base de datos, **modificar** una huella de versión y **borrar** filas de la auditoría.
- **Qué significa.** La propia base rechaza ambas: «tabla append-only: UPDATE no permitido...» y «DELETE no permitido...». La traza es **inalterable**, incluso para quien tenga acceso a la base de datos con el usuario de la aplicación.

Conclusión — evidencia del criterio 7

- 4 versiones consecutivas, con operación, actor, instante y huella.
- Estado reconstruido en cada instante (incluido «no existía»).
- Estado terminal respetado (409).
- Inalterabilidad comprobada con la base de datos.

24. Pruebas automáticas

24.1 Ejecutar las pruebas de la ERSO 008

```
$ cd /home/geovani/Descargas/generic/bitacora/new_proyect
$ TEST_DB_URL=jdbc:postgresql://127.0.0.1:55432/wallet_test ./gradlew :vdr-service:test --tests '*Erso008*' --rerun-tasks -q 2>&1 | tail -3; echo '(sin salida = todas pasaron)'
(sin salida = todas pasaron)
$ python3 - <<'EOF'
> import re, glob, html
> f = glob.glob('vdr-service/build/test-results/test/*Erso008*.xml')[0]
> s = open(f, encoding='utf-8').read()
> tot = re.search(r'tests="(\d+)" skipped="(\d+)" failures="(\d+)"', s).groups()
> print(f"resultado: {tot[0]} pruebas, {tot[1]} omitidas, {tot[2]} fallos\n")
> for m in re.finditer(r'<testcase name="([^\"]+)"[^>]*?(/>|>(.*)</testcase>)', s, re.S):
>     name = html.unescape(m.group(1)).removesuffix("(")
>     estado = "FALLA" if "<failure" in (m.group(3) or "") else "PASA "
>     print(f" [{estado}] {name}")
> EOF
resultado: 8 pruebas, 0 omitidas, 0 fallos

[PASA ] criterio 4 - la escritura exige version esperada y devuelve version hash y URL publica
[PASA ] prueba de posesion - clave ajena, hash distinto y rotacion
[PASA ] criterio 1 - si falla una precondition no se emite desafio
[PASA ] criterio 3 - el documento no incluye claves privadas y id y controladores corresponden al
recurso
[PASA ] criterio 5 y 6 - confirmacion por lectura y hash, y operacion pendiente si no hay respuesta
[PASA ] idempotencia - reintentar con la misma clave no duplica la version
[PASA ] criterio 7 - la traza permite reconstruir el estado en cada momento
[PASA ] criterio 2 - el desafio es de un solo uso con tipo y audiencia especificos
```

- **Qué hace.** Ejecuta `Erso008LifecycleTest` en una base de datos desechable (`vdr-test-pg`, puerto 55432).
- **Qué significa.** Ocho pruebas, ninguna falla:

Prueba	Criterio
criterio 1 - si falla una precondition no se emite desafio	1
criterio 2 - el desafio es de un solo uso con tipo y audiencia especificos	2
criterio 3 - el documento no incluye claves privadas...	3
criterio 4 - la escritura exige version esperada y devuelve version hash y URL publica	4
idempotencia - reintentar con la misma clave no duplica la version	4
prueba de posesion - clave ajena, hash distinto y rotacion	2 y 4
criterio 5 y 6 - confirmacion por lectura y hash, y operacion pendiente si no hay respuesta	5 y 6
criterio 7 - la traza permite reconstruir el estado en cada momento	7

24.2 El recorrido de extremo a extremo (extracto del registro)

El script `scripts/e2e.sh` ejecuta todo el proyecto (121 comprobaciones). Este es el bloque de la ERSO 008:

```

ERSO 2026-008 · Ciclo de vida y trazabilidad (namespace entidades/avance-ciclo)

Clave P-256 generada. Multikey público: zDnaecetVC6EQRZz6jwBpD2yRezRjoauvSHWsguvSpNGXEKK7
Clave P-256 generada. Multikey público: zDnaeU8qaBcG4Hk4o9PNDgXfSJzVwZrauzXCTFiYk3X3NsWpV
- versión 1: CREATE -
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío ..... 201 id=958618fe-1a9d-462a-9449-7ae815c98338 audience=vdr:civica-
desarrollo.avance.org.co:did-operation purpose=CREATE
[3/4] prueba ..... ES256 kid=did:web:civica-desarrollo.avance.org.co:entidades:avance-ciclo#key-1
docHash=sha256:7909c893d0d82e5b7cb376d173d19c9e6620e0f9449b3206747f...
[4/4] escritura .... 201 If-Match=0 Idempotency-Key=68b91e32-62e5-443e-ac6c-4e4c7b4793e1
{
  ✓ PASS C4: la escritura devolvió versión, hash y URL pública
  ✓ PASS C5: confirmada por lectura de la URL y comparación de hash
  - versión 2: UPDATE con rotación de clave (firma la clave VIGENTE c1) -
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío ..... 201 id=470c36a2-6727-4d38-8239-e83853b026ed audience=vdr:civica-
desarrollo.avance.org.co:did-operation purpose=UPDATE
[3/4] prueba ..... ES256 kid=did:web:civica-desarrollo.avance.org.co:entidades:avance-ciclo#key-1
docHash=sha256:83d7bfea0ecebdbc2038381891c04e27c09e7c49a884afb77a...
[4/4] escritura .... 200 If-Match=1 Idempotency-Key=a0e14790-7a0a-4807-a95d-5d8f101d9d03
{
  ✓ PASS rotación aceptada: la prueba la firmó la clave vigente
  "error": "INVALID_PROOF",
  "message": "Prueba de posesión inválida: firma no válida"
}
  ✓ PASS la clave anterior ya no puede escribir (403 INVALID_PROOF)
  "currentVersion=2"
}
  ✓ PASS C4: versión esperada equivocada se rechaza (412 VERSION_CONFLICT)
  - versión 3: UPDATE (firma c2) -
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío ..... 201 id=f1a7b890-7344-4c84-9b69-502ec3c013ab audience=vdr:civica-
desarrollo.avance.org.co:did-operation purpose=UPDATE
[3/4] prueba ..... ES256 kid=did:web:civica-desarrollo.avance.org.co:entidades:avance-ciclo#key-1
docHash=sha256:eb337892c97281c6a189a5231eda641efe3883c431cb7ealed02...
[4/4] escritura .... 200 If-Match=2 Idempotency-Key=9d04e3bf-4ca0-4c4f-9181-c92e04c189e6
{
  ✓ PASS versión 3 publicada

```

```

- versión 4: DEACTIVATE -
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío ..... 201 id=5a2d3beb-b2f1-4d73-911c-955565f96dc3 audience=vdr:civica-
desarrollo.avance.org.co:did-operation purpose=DEACTIVATE
[3/4] prueba ..... ES256 kid=did:web:civica-desarrollo.avance.org.co:entidades:avance-ciclo#key-1
docHash=sha256:eb337892c97281c6a189a5231eda641efe3883c431cb7ea1ed02...
[4/4] escritura .... 200 If-Match=3 Idempotency-Key=e0924ccf-c437-4044-8e13-9d2336f54f5e
{
  ✓ PASS desactivación confirmada (la URL pública responde 410)
  ✓ PASS lectura pública del DID desactivado -> 410 Gone
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío FALLÓ: HTTP 409 {"error":"TERMINAL_STATE","message":"El historial está desactivado; se
requiere una nueva instancia (otro namespace)"}
  ✓ PASS regla transversal: un historial terminal no vuelve a activarse (409 TERMINAL_STATE)
  Historial de versiones (C7):
    v1 CREATE 2026-09-30T14:21:26.800033Z
    sha256:7909c893d0d82e5b7cb376d173d19c9e6620e0f9449b3206747f7ea927af0d77
    v2 UPDATE 2026-09-30T14:21:30.817406Z
    sha256:83d7bfea0ecebdbdcb2038381891c04e27c09e7c49a884afb77aaef417d1d6f0
    v3 UPDATE 2026-09-30T14:21:36.790936Z
    sha256:eb337892c97281c6a189a5231eda641efe3883c431cb7ea1ed0298cfdadbe0f4
    v4 DEACTIVATE 2026-09-30T14:21:38.868470Z
    sha256:eb337892c97281c6a189a5231eda641efe3883c431cb7ea1ed0298cfdadbe0f4
    ✓ PASS C7: 4 versiones en orden CREATE, UPDATE, UPDATE, DEACTIVATE
    ✓ PASS C7: estado reconstruido en el instante de la v2 = ACTIVE, versión 2
    ✓ PASS C7: estado reconstruido en el instante de la v4 = DEACTIVATED
    ✓ PASS C7: el historial no se puede reescribir (trigger append-only en PostgreSQL)
    ✓ PASS C2: desafíos emitidos quedaron registrados (tipo y audiencia)
    ✓ PASS C1: sin certificado cliente no se emite desafío (nginx 400; la aplicación ni se entera)
    ✓ PASS C1: la denegación por precondiciones (namespace ajeno) dejó rastro y no creó desafío

```

24.3 Limpieza

Las instancias temporales (`vdr-lento` y `confirm-lab`) se eliminan:

```

$ cd /home/geovani/Descargas/generic/bitacora/new_proyect/deploy; docker rm -f vdr-lento confirm-lab;
rm -rf ../evidencias/work/confirm008
vdr-lento
confirm-lab

```

Los datos de laboratorio permanecen en la base (los DID `ciclo-008-...` y `pend-...`); por diseño, **las versiones y la auditoría no se pueden borrar**.

PARTE V — PREGUNTAS Y RESPUESTAS

25. Preguntas sobre los conceptos

- 1. ¿Por qué un desafío debe ser de un solo uso?** Para evitar la **repetición**: si alguien captura una petición firmada, no puede reenviarla, porque el desafío ya se gastó. Además, prueba que quien firma posee la clave **ahora**, no que la tuvo alguna vez.
- 2. ¿Por qué la prueba firma el desafío y la huella del documento?** El desafío ata la firma al **momento**; la huella la ata al **contenido**. Sin la huella, alguien podría tomar una prueba válida y adjuntarla a un documento distinto. Sin el desafío, podría reutilizar una prueba antigua.
- 3. ¿Por qué firma la clave vigente y no la del documento nuevo?** Porque si bastara la clave del documento propuesto, cualquiera podría publicar un documento con **su** clave y tomar el control del DID. La clave vigente es la que **ya** gobierna el DID; en la creación no hay historia, por eso allí firma la del propio documento.
- 4. ¿Qué es la rotación de claves y por qué es un UPDATE ?** Cambiar la clave del documento por otra. Es un **UPDATE** cuya prueba firma la clave vieja y cuyo documento nuevo contiene la clave nueva. A partir de la nueva versión, la clave vieja deja de tener poder de escritura.
- 5. ¿Qué problema resuelve If-Match ?** La **escritura simultánea**: dos clientes leen la versión 1 y ambos quieren cambiarla. El primero gana y crea la 2; al segundo se le responde **412 VERSION_CONFLICT**: tiene que volver a leer y decidir. Así nadie pisa sin saberlo el trabajo de otro.
- 6. ¿Qué problema resuelve la clave de idempotencia?** La **repetición por fallo de red**: el cliente no sabe si su petición llegó. Si la repite con la misma clave, el registro devuelve el resultado original y **no crea otra versión**. Es distinto de **If-Match**: aquél protege contra *otro* escritor; éste, contra *uno mismo* repitiéndose.
- 7. ¿Por qué la idempotencia se revisa antes de consumir el desafío?** Porque el reintento ya trae un desafío gastado; si se revisara después, un reintento legítimo fallaría. Así el reintento recibe el resultado original.
- 8. ¿Por qué un desafío se gasta aunque la operación falle?** Es el significado de «un solo uso»: cualquier intento lo consume. Si no, un atacante podría probar variantes con el mismo desafío. El costo es que, tras un fallo, hay que pedir otro.
- 9. ¿Qué diferencia hay entre PENDING y CONFIRMED ?** **CONFIRMED**: el registro leyó la URL pública y el hash coincidió. **PENDING**: no se pudo comprobar a tiempo; el documento está guardado, pero **no se da por publicado**.
- 10. ¿Qué hace reconcile ?** Vuelve a leer la URL: si coincide, la operación pasa a **CONFIRMED**; si entre tanto hay una versión posterior, queda **SUPERSEDED**.
- 11. ¿Por qué un DID desactivado no se reactiva?** Porque otros pudieron confiar en la baja. Reactivarlo permitiría que una identidad «muerta» reviva, quizá en otras manos. Si hace falta, se

crea una **instancia nueva** (otro namespace).

12. ¿Qué significa «solo-agregar»? Que la tabla solo admite inserciones; la base de datos rechaza `UPDATE` y `DELETE` mediante disparadores. Es la garantía de que la historia no se reescribe.

13. ¿Qué advierte la ERSO sobre la rotación? Que **no ofrece por sí sola validación histórica**: verificar una firma antigua exige conocer la clave de su época; si el consumidor solo mira el documento actual, la firma vieja se rechaza. El historial lo permitiría, pero hoy no se usa para verificar (§29).

14. ¿Y sobre la pérdida de clave? Que **guardar un documento público no la resuelve**: es público, no devuelve el control. Requiere otro mecanismo (respaldo, custodia, autoridad de recuperación). Aquí no se inventó; la consecuencia es desactivar y crear otra instancia.

15. ¿Por qué `DEACTIVATE` conserva el hash de la última versión? Porque no hay documento nuevo; la prueba firma el hash del último estado, y la versión de baja lo repite como constancia de «qué estaba vigente cuando se dio de baja».

26. Preguntas por laboratorio y por criterio

Criterio 1 (laboratorio A)

P1. *Siete situaciones incorrectas: ¿cuántos desafíos nuevos?* → **Cero**. El contador queda igual. Y seis filas `CHALLENGE_DENIED` (el séptimo caso, sin certificado, lo frena nginx).

P2. *¿Por qué `412` y no `401/403`?* → Porque la identidad puede ser válida; lo que falla es una **precondición** (espacio ajeno, ruta sin reservar...).

P3. *¿Qué demuestra el caso «cuenta deshabilitada»?* → Que deshabilitar una cuenta tiene **efecto inmediato**, aun con token vigente.

Criterio 2 (laboratorio B)

P4. *¿Qué campos tiene un desafío y para qué sirve cada uno?*

Campo	Para qué
<code>challengeId</code>	Identificarlo al usarlo
<code>nonce</code>	El valor aleatorio que se firma
<code>did</code>	A qué DID corresponde
<code>purpose</code>	Tipo: <code>CREATE</code> / <code>UPDATE</code> / <code>DEACTIVATE</code>
<code>audience</code>	Para qué sistema
<code>expiresAt</code>	Cuándo caduca

P5. *Segundo uso de un desafío: ¿qué pasa?* → `403 CHALLENGE_INVALID`. **P6.** *Desafío `UPDATE` usado para `DEACTIVATE`:* → `403 CHALLENGE_MISMATCH`. **P7.** *Desafío vencido:* → `403 CHALLENGE_INVALID`. **P8.** *Prueba con otra audiencia:* → `403 INVALID_PROOF`.

P9. ¿Dónde está «el registro de emisión»? → En `audit_log`, evento `CHALLENGE_ISSUED` (identificador, tipo, audiencia, vencimiento), y en la tabla `challenges`.

Criterio 3 (laboratorio C)

P10. Cambiar solo el `id` rompió tres reglas, ¿por qué? → Porque la clave de verificación y su controlador están definidos en función del `id`; el validador revisa la coherencia completa (`ID_MISMATCH`, `INVALID_VERIFICATION_METHOD`, `UNAUTHORIZED_CONTROLLER`). **P11.** ¿Se guardó algo? → No: cero documentos.

Criterio 4 (laboratorio D)

P12. Tabla de respuestas:

Situación	Código
Sin If-Match	428 IF_MATCH_REQUIRED
Versión errónea	412 VERSION_CONFLICT
Versión correcta	200 (201 si crea)
Repetición idéntica	200 con <code>replayed:true</code>
Misma clave, otro contenido	422 IDEMPOTENCY_KEY_REUSED
Clave vieja tras rotar	403 INVALID_PROOF

P13. ¿Qué tres datos devuelve una escritura correcta? → Versión, hash y URL pública (además del identificador de operación y el estado). **P14.** ¿Por qué el hash que devuelve el registro coincide con el calculado a mano? → Ambos calculan SHA-256 del mismo JSON canónico.

Criterio 5 (laboratorio E)

P15. ¿Qué hashes se comparan? → El de la evidencia (operación), el del documento servido por la URL pública, y el `X-Content-Hash`. Los tres coinciden.

Criterio 6 (laboratorio F)

P16. Si la confirmación falla, ¿se deshace la escritura? → No: queda guardada, pero la operación es `PENDING` (202), no «publicada». **P17.** ¿Por qué el primer `reconcile` con el servicio ya restablecido podría fallar? → Por la memoria de nombres de la JVM (~10 s); no es un fallo de la ERSO, sino de cómo se montó la prueba. Por eso se esperan 15 s. **P18.** ¿Por qué hizo falta una segunda instancia? → Porque el registro real confirma contra su dirección pública, que funciona; para provocar un fallo sin romper el sistema se usó otra instancia con la confirmación apuntando a un nombre inexistente.

Criterio 7 (laboratorio G)

P19. ¿Cuántas versiones y con qué hash la de baja? → Cuatro; la de baja repite el hash de la v3. **P20.** ¿Qué responde `state?at=` para un instante anterior a la creación? → 404 «no existía». **P21.** ¿Qué ve el público de un DID desactivado? → 410 Gone. **P22.** ¿Se puede reactivar? → No: 409 `TERMINAL_STATE`; hace falta otra instancia (otro namespace). **P23.** ¿Cómo se sabe que el historial

no se puede reescribir? → Porque la base rechaza UPDATE / DELETE en las tablas de versiones y de auditoría.

Pruebas automáticas

P24. *¿Cuántas pruebas hay?* → Ocho, todas en verde.

PARTE VI — CÓMO RESPONDER A CADA CRITERIO

27. Redacción modelo de las siete respuestas

Criterio 1 — Las precondiciones se cumplen; si alguna falla no se emite desafío

Antes de emitir un desafío, el registro evalúa: cuenta habilitada, canal restringido (certificado válido de la misma entidad), DID bien formado y del dominio, ruta reservada y namespace propio. Se comprobaron siete incumplimientos (espacio ajeno, ruta sin reservar, dominio ajeno, DID inválido, certificado de otra entidad, cuenta deshabilitada y ausencia de certificado): ninguno produjo desafío (el contador no cambió) y los seis que llegaron a la aplicación quedaron en la auditoría con su motivo. **Evidencia:** §17; prueba `criterio 1 - si falla una precondicion no se emite desafio`.

Criterio 2 — Desafío de un solo uso, con tipo y audiencia

El desafío incluye tipo (`purpose`), audiencia (`vdr:<dominio>:did-operation`) y vigencia de 300 s, y su emisión queda registrada (`CHALLENGE_ISSUED`). Se consume de forma atómica: un segundo uso da `CHALLENGE_INVALID` ; usado para otro tipo, `CHALLENGE_MISMATCH` ; vencido, `CHALLENGE_INVALID` ; con prueba para otra audiencia, `INVALID_PROOF` . **Evidencia:** §18.

Criterio 3 — Documento sin claves privadas, con `id` y controladores del recurso

El validador de publicación rechaza (422) documentos con material de clave privada, con `id` distinto del DID (y la incoherencia que acarrea), con controlador ajeno o con datos civiles, y no deja ningún registro. **Evidencia:** §19.

Criterio 4 — Versión esperada; devuelve versión, hash y URL

La escritura exige `If-Match` (428 si falta; 412 `VERSION_CONFLICT` con `currentVersion` si no coincide) y `Idempotency-Key` . Una escritura correcta devuelve versión, hash y URL pública. Un reintento idéntico devuelve la misma operación con `replayed:true` sin crear versión; la misma clave con otro contenido da `IDEMPOTENCY_KEY_REUSED` . La prueba de posesión se verifica con la clave vigente: tras la rotación, la clave anterior da `INVALID_PROOF` . **Evidencia:** §20.

Criterio 5 — Confirmación por lectura y hash

Tras escribir, el registro lee la URL pública y compara el hash; si coincide, la operación pasa a `CONFIRMED` con `confirmedAt`. El hash de la evidencia, el del documento servido y el `X-Content-Hash` son idénticos. **Evidencia:** §21; prueba criterio 5 y 6.

Criterio 6 — Pendiente si no hay respuesta en plazo

Cuando la lectura de confirmación falla, la operación queda `PENDING` (HTTP 202) con `PUBLICATION_PENDING` y causa, y no se da por publicada; reconcila la mantiene pendiente mientras el servicio siga inaccesible y la pasa a `CONFIRMED` («hash coincide») cuando se restablece. **Evidencia:** §22.

Criterio 7 — La traza reconstruye el estado en cada momento

Cada operación añade una versión (operación, actor, instante, hash) a `did_document_versions`, tabla solo-agregar. El estado se reconstruye en cada instante (`ACTIVE` v1, v2, v3; `DEACTIVATED` v4; «no existía» antes de la creación); el DID desactivado se sirve como 410 y no se reactiva (`409 TERMINAL_STATE`); la base rechaza `UPDATE/DELETE` sobre versiones y auditoría. **Evidencia:** §23.

28. Lista de comprobación para quien acepta

☐ Qué comprobar	Cómo	Resultado esperado
☐ Precondiciones	7 casos del laboratorio A	Motivo distinto en cada uno; contador igual
☐ Auditoría de denegaciones	<code>audit_log</code> <code>CHALLENGE_DENIED</code>	Una fila por caso con <code>failures</code>
☐ Desafío con tipo y audiencia	<code>NUEVO \$DID CREATE</code>	<code>purpose</code> , <code>audience</code> , <code>expiresAt</code>
☐ Registro de emisión	<code>audit_log</code> <code>CHALLENGE_ISSUED</code>	Presente
☐ Un solo uso	Reusar el desafío	<code>403 CHALLENGE_INVALID</code>
☐ Tipo	<code>UPDATE</code> usado para <code>DEACTIVATE</code>	<code>403 CHALLENGE_MISMATCH</code>
☐ Vigencia	Desafío vencido	<code>403 CHALLENGE_INVALID</code>
☐ Audiencia	Prueba con otra <code>aud</code>	<code>403 INVALID_PROOF</code>
☐ Documentos prohibidos	4 variantes	<code>422</code> y 0 filas
☐ If-Match ausente	<code>PUT</code> sin cabecera	<code>428</code>
☐ Versión errónea	<code>If-Match: 5</code>	<code>412 VERSION_CONFLICT</code>
☐ Respuesta correcta	<code>If-Match: 1</code>	versión, hash, <code>publicUrl</code>
☐ Idempotencia	Repetir	<code>replayed: true</code>

☐ Qué comprobar	Cómo	Resultado esperado
☐ Clave reutilizada	Mismo Idempotency-Key , otro contenido	422 IDEMPOTENCY_KEY_REUSED
☐ Clave rotada	Firmar con la clave vieja	403 INVALID_PROOF
☐ Confirmación	Operación + comparar hashes	CONFIRMED , hashes idénticos
☐ Pendiente	Confirmación inalcanzable	202 PENDING
☐ Reconciliar	Con el servicio de vuelta	CONFIRMED
☐ Historial	GET /versions	4 versiones
☐ Estado por instante	GET /state?at=	ACTIVE / DEACTIVATED /404
☐ Estado terminal	CREATE tras la baja	409 TERMINAL_STATE
☐ Inalterable	UPDATE / DELETE en tablas	Error append-only
☐ Pruebas automáticas	./gradlew :vdr-service:test --tests '*Erso008*'	8/8

PARTE VII — LÍMITES, HALLAZGOS Y OPERACIÓN

29. Lo que este informe NO demuestra, y lo que se encontró

29.1 Los dos límites que la propia ERSO advierte

Advertencia de la ERSO	Estado
Rotar el documento no ofrece validación histórica	No se ofrece. El historial permite reconstruir el estado, pero el consumidor (ERSO 007) valida contra el documento actual : una firma de una clave ya rotada se rechaza. Verificar «con la clave de su época» requeriría que el consumidor consultara el historial, que no es público.
La recuperación ante pérdida de clave no se resuelve guardando un documento público	No se resuelve. Si se pierde la clave privada no hay forma de actualizar ni desactivar el documento (la prueba de posesión exige la clave vigente). Habría que definir un mecanismo aparte (custodia en HSM, respaldo fuera de línea, autoridad de recuperación).

29.2 Otros límites

Límite	Detalle
Pruebas propias	La prueba de posesión es un JWS ES256 con un contenido fijo de cinco campos; no es una <i>Data Integrity Proof</i> de W3C. Se documenta para quien integre.
Historial no público	Solo lo consulta la entidad dueña del namespace o la administración; no hay endpoint público.
Desafío gastado en cada intento	Tras un fallo hay que pedir otro desafío. Intencional.
Concurrencia real	Se demostró el efecto de If-Match (rechazo por versión), no una carrera real entre dos procesos escribiendo a la vez; el bloqueo de la fila del DID la cubre en el diseño y en la prueba automática.
Atajos de laboratorio	Se adelantó el vencimiento de un desafío y se deshabilitó/habilitó una cuenta modificando la base de datos; ambos son recursos para no esperar y se deshacen en el mismo paso.
Segunda instancia	vdr-lento comparte base con el registro real y se publica solo en 127.0.0.1. Es un montaje de prueba, no una topología de producción.
Memoria de nombres	La JVM recuerda ~10 s los nombres inexistentes; condiciona cuándo se puede reconciliar tras restablecer un servicio.
Secretos	Los de laboratorio están en <code>deploy/.env</code> (excluido de git).
Sin limitación de tasa	No hay bloqueo por intentos fallidos.

29.3 Hallazgo heredado: HEAD responde 404

Descubierto en la ERSo 005; no afecta a esta ERSo (la confirmación usa GET).

30. Si algo no sale como en el informe

Síntoma	Causa probable	Qué hacer
jq: parse error en NUEVO	El token está vacío (credenciales o certificado)	Repetir <code>TK=\$(TOKEN ...)</code> ; revisar <code>.env</code> cargado
403 CHALLENGE_INVALID inesperado	Se reutilizó o venció el desafío	Pedir uno nuevo con NUEVO
403 INVALID_PROOF inesperado	Se firmó con la clave equivocada, con otra huella o con otro propósito	Revisar los argumentos de PRUEBA y que H sea la huella del documento enviado
422 IDEMPOTENCY_KEY_REUSED inesperado	Se repitió la clave con otro contenido	Usar una clave nueva por intención
412 VERSION_CONFLICT	If-Match no es la versión actual	Consultar <code>currentVersion</code> y repetir
El reconcile sigue en PENDING	La JVM aún recuerda el fallo de nombre, o confirm-lab no está arriba	Esperar 15 s; <code>docker ps</code>
<code>docker: network vdr-ssi_default not found</code>	Proyecto con otro nombre	<code>docker network ls</code> y ajustar
vdr-lento ya existe	Un laboratorio anterior no terminó	<code>docker rm -f vdr-lento confirm-lab</code>
412 NAMESPACE_NOT_OWNED al publicar	Falta el espacio <code>entidades/*</code>	Ver informe de la ERSo 006, §12.5

31. Operación diaria

```
cd deploy
docker compose up -d           # levantar
docker compose --profile tools down # bajar (NUNCA con -v: borra la base)
bash scripts/lab-008-completo.sh # repetir todos los laboratorios
```

- **Consultar el historial de un DID:** `GET /admin/v1/documents/{did}/versions` (entidad dueña o administración).
- **Resolver una operación pendiente:** `POST /admin/v1/operations/{id}/reconcile`.
- **Dar de baja una entidad:** operación `DEACTIVATE` (terminal).

ANEXOS

Anexo A — Los scripts

Script	Para qué sirve
scripts/lab-008-completo.sh	Ejecuta los laboratorios A a H, con la segunda instancia temporal y la limpieza
scripts/e2e.sh	Recorrido de extremo a extremo (121 comprobaciones)
scripts/gen-dev-certs.sh	Genera la CA y los certificados de laboratorio
scripts/generar-pdf.sh	Convierte un informe a PDF

Anexo B — Autocomprobación (con respuestas)

1. ¿Qué cinco campos firma la prueba de posesión? → `aud`, `challenge`, `did`, `docHash`, `purpose`.
2. ¿Qué clave firma en un `UPDATE`? → La vigente (la del documento actual).
3. ¿Y en un `CREATE`? → La del propio documento (no hay historia).
4. ¿Cuánto dura un desafío? → 300 s.
5. ¿Qué ocurre con `If-Match` ausente? → 428.
6. ¿Qué significa `replayed: true`? → Es el resultado de una operación anterior, repetida por idempotencia.
7. ¿Qué código HTTP indica «pendiente»? → 202.
8. ¿Qué ve el público de un DID desactivado? → 410.
9. ¿Qué responde un `CREATE` sobre un DID desactivado? → 409 `TERMINAL_STATE`.
10. ¿Qué impide modificar el historial? → Disparadores `forbid_mutation` en la base.

Anexo C — Ejercicios del método (con respuesta modelo)

Ejercicio 1. Descompón el criterio 4. → Verbos: *exige* (versión esperada) y *devuelve* (versión, hash, URL). Evidencia: respuesta del registro. Casos negativos: sin versión, versión errónea, repetición. Mecanismos: `If-Match`, bloqueo de fila, idempotencia.

Ejercicio 2. Diseña una prueba para «un desafío de `UPDATE` no sirve para `DEACTIVATE`». → Pedir un desafío `UPDATE`, firmar una prueba con `purpose: DEACTIVATE` y enviarla a `/deactivate`: debe dar `CHALLENGE_MISMATCH` (laboratorio B, §18.7).

Ejercicio 3. ¿Cómo demostrarías que el pasado no se puede reescribir? → Intentar `UPDATE` sobre `did_document_versions` y `DELETE` sobre `audit_log`: la base debe rechazarlo (§23.8).

Ejercicio 4. Un cliente repite una escritura por un corte de red. ¿Qué evitas con la idempotencia y con `If-Match`? → Idempotencia: evita duplicar la versión al repetir la **misma** intención. `If-Match`: evita pisar el cambio de **otro** escritor.

Anexo D — Referencias

- W3C — Decentralized Identifiers (DIDs) v1.1 (operaciones del método).
- W3C — DID Resolution v1.
- RFC 9110 (semántica HTTP: `If-Match`, `ETag`, 412, 428).
- RFC 7515 (JWS).
- Patrones de claves de idempotencia para escrituras.
- `docs/VERSIONES-NORMATIVAS.md`, `docs/MARCO-CONCEPTUAL.md`, `docs/COMO-SE-RESOLVIO-CADA-CRITERIO.md`.